ORNL/XX-XXXX/XXX

User Manual - HydraGNN v5.0: Distributed Implementation of Multi-Tasking Graph Neural Networks

Massimiliano Lupo Pasini
Jong Youl Choi
Kshitij Mehta
Pei Zhang
Rylie Weaver
Richard Messerly
Arindam Chowdhury
Adithya Raman
Ashwin M. Aji

Draft. Document has not been reviewed and approved for public release.

April 2026

OAK RIDGE National Laboratory

ORNL IS MANAGED BY UT-BATTELLE LLC FOR THE US DEPARTMENT OF ENERGY

ORNL/XX-XXXX/XXX

Computing and Computational Sciences Directorate
Computational Sciences and Engineering Division
Computer Science and Mathematics Division

USER MANUAL - HYDRAGNN V5.0: DISTRIBUTED IMPLEMENTATION OF MULTI-TASKING GRAPH NEURAL NETWORKS

Massimiliano Lupo Pasini
Jong Youl Choi
Kshitij Mehta
Pei Zhang
Rylie Weaver
Richard Messerly
Arindam Chowdhury
Adithya Raman
Ashwin M. Aji

April 2026

Prepared by
OAK RIDGE NATIONAL LABORATORY
Oak Ridge, TN 37831
managed by
UT-BATTELLE LLC
for the
US DEPARTMENT OF ENERGY
under contract DE-AC05-00OR22725

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

ASE	Atomic Simulation Environment
CGCNN	Crystal Graph Convolutional Neural Network
DDP	distributed data parallelism
DL	deep learning
DOE	US Department of Energy
EGNN	E(n) Equivariant Graph Neural Network
FC	fully connected
FSDP	Fully Sharded Data Parallelism
GCNN	graph convolutional neural network
GNN	graph neural network
GraphGPS	General, Powerful, and Scalable Graph Transformer
HPC	high-performance computing
HPO	hyperparameter optimization
LPE	Laplacian positional encoding
MLIP	machine-learned interatomic potential
MPNN	Message-Passing Neural Network
MTL	multi-task learning
NLL	negative log-likelihood
PAINN	Polarizable Atom Interaction Neural Network
PBC	periodic boundary conditions
PyG	PyTorch-Geometric
RBF	radial basis function

FOREWORD

PREFACE

This manual is organized to serve different audiences:

- **New users**: Start with the Installation (Section 3) and Quick Start, which includes a minimal working example. Then proceed to Complete Examples (Section 15) for end-to-end workflows before consulting the detailed sections.
- **Configuration reference**: See the Definition of the GNN Model (Section 5) and Appendix B for all supported options.
- **Distributed training**: See Section 13 for DDP, DeepSpeed, FSDP, DDStore, and profiling.
- **Custom models and datasets**: See Modules (Section 11) and Dataset Classes (Section 14).

1. ACKNOWLEDGEMENTS

This work was supported in part by the Office of Science of the Department of Energy and by the Artificial Intelligence Initiative as part of the Laboratory Directed Research and Development (LDRD) Program of Oak Ridge National Laboratory (ORNL), managed by UT-Battelle, LLC, for the US Department of Energy under contract DE-AC05-00OR22725. This work was supported in part by the US-DOE Advanced Scientific Computing Research (ASCR) under contract DE-SC0023490.

ABSTRACT

This document serves as the user manual for HydraGNN v5.0, a scalable graph neural network (GNN) architecture for simultaneous prediction of multiple target properties using multi-task learning (MTL). This version of HydraGNN has been developed primarily to support the development, training, and deployment of predictive graph-based deep learning (DL) models for atomistic materials modeling. HydraGNN is templated over 13 message-passing policies, including invariant models (GIN, PNA, PNAPlus, GAT, MFC, CGCNN, SAGE, SchNet, DimeNet) and equivariant models (EGNN, PNAEq, PAINN, MACE), and supports distributed training via distributed data parallelism (DDP), DeepSpeed, and Fully Sharded Data Parallelism (FSDP) on leadership-class supercomputers. **Although HydraGNN can be applied to problems beyond atomistic materials modeling, its current use is confined to homogeneous graphs.** Additional capabilities include machine-learned interatomic potentials with energy-conserving forces, General, Powerful, and Scalable Graph Transformer (GraphGPS) global attention, periodic boundary conditions, hyperparameter optimization, mixed-precision training, and uncertainty quantification.

2. MOTIVATION AND BACKGROUND

2.1 INTRODUCTION

Graph neural networks (GNNs) have emerged as a powerful class of deep learning (DL) models for learning on graph-structured data. In scientific computing, graphs naturally represent atomistic structures, with atoms as nodes and chemical bonds or proximity relationships as edges, making GNNs well suited for predicting scientifically relevant properties such as total energy, interatomic forces, band gaps, and spectra.

This release of HydraGNN is aimed primarily at the development, training, and deployment of predictive graph-based DL models for atomistic materials modeling. While many of the underlying algorithms can also be useful in other scientific and engineering applications, the current software stack is designed around homogeneous graph data structures and does not target heterogeneous-graph workflows.

Scope Limitation

HydraGNN currently supports homogeneous graphs only. Although the framework can be adapted to application domains beyond atomistic materials modeling, this version of the software does not target heterogeneous-graph problems with multiple node or edge types requiring distinct message-passing semantics.

However, practical deployment of GNNs at scale presents several challenges: (i) selecting the optimal message-passing policy for a given dataset and application, (ii) simultaneously predicting multiple correlated properties to exploit physics-based relationships, and (iii) scaling training to leadership-class supercomputers to handle the large datasets required for accurate surrogate models.

HydraGNN addresses these challenges through a unified, modular framework that combines multi-task learning (MTL) with a templated architecture supporting 13 interchangeable message-passing policies. Named after the multi-headed Hydra of Greek mythology, HydraGNN forks its architecture into separate prediction heads, one for each target property, while sharing a common set of message-passing layers that learn a joint representation of the input graph.

2.2 DISTINCT CAPABILITIES OF HYDRAGNN

HydraGNN offers the following distinct capabilities that differentiate it from other GNN frameworks:

- **Multi-task learning:** HydraGNN supports the simultaneous prediction of an arbitrary number of target properties, both at the node level (e.g., atomic forces, partial charges) and at the graph level (e.g., total energy, band gap), using a unified multi-headed architecture with configurable task weights.
- **13 interchangeable message-passing policies:** The framework supports GIN, PNA, PNAPlus, GAT, MFC, Crystal Graph Convolutional Neural Network (CGCNN), SAGE, SchNet, DimeNet, E(n) Equivariant Graph Neural Network (EGNN), PNAEq, Polarizable Atom Interaction Neural Network (PAINN), and MACE. An object-oriented design allows these to be swapped as hyperparameters via a single configuration change, enabling systematic comparisons across architectures.
- **Equivariant architectures:** Four of the supported models (EGNN, PNAEq, PAINN, and MACE) provide E(n) or O(3) equivariance, enabling physically consistent predictions of vectorial and tensorial quantities such as interatomic forces.

- **Machine-learned interatomic potentials:** HydraGNN supports energy-conserving force predictions via automatic differentiation of the total energy with respect to atomic positions ($\mathbf{F}_i = -\nabla_{\mathbf{r}_i} E$), enabling the construction of machine-learned interatomic potentials (MLIPs) with thermodynamic consistency.
- **Distributed training at scale:** HydraGNN provides native support for distributed data parallelism (DDP), DeepSpeed with ZeRO optimization, and Fully Sharded Data Parallelism (FSDP) (v1 and v2), with automatic backend detection (NCCL, Intel CCL, GLOO) and support for SLURM and LSF job schedulers.
- **Global attention via GraphGPS:** The framework integrates the General, Powerful, and Scalable Graph Transformer (GraphGPS) architecture, which augments any local message-passing layer with global multi-head self-attention for capturing long-range interactions.
- **Periodic boundary conditions:** Native support for periodic boundary conditions (PBC) using the `vesin` library for efficient neighbor-list construction with periodic images, critical for modeling crystalline materials.
- **Flexible data pipelines:** HydraGNN supports multiple raw data formats (LSMS, CFG, XYZ), serialized formats (Pickle, ADIOS2), and distributed data access via DDStore, with configurable feature normalization, compositional stratified splitting, and atomic descriptor embeddings.
- **Hyperparameter optimization:** Integration with DeepHyper and Optuna enables automated hyperparameter search across architecture hyperparameters, training parameters, and message-passing policy selection.
- **Mixed-precision training:** Support for `bf16`, `fp32`, and `fp64` precision with automatic mixed-precision casting for memory-efficient training on modern accelerators.
- **Uncertainty quantification:** Through the Gaussian negative log-likelihood (NLL) loss function, HydraGNN can output prediction variances alongside mean predictions for uncertainty-aware inference.
- **Comprehensive testing:** An extensive test suite with continuous integration ensures correctness across all supported models, loss functions, optimizers, and distributed configurations.

3. INSTALLATION

This user manual documents the version of HydraGNN corresponding to the `v5.0` release tag, available at the ORNL GitHub repository: https://github.com/ORNL/HydraGNN/releases/tag/v5.0.

3.1 REQUIREMENTS

HydraGNN is a Python package built on top of PyTorch and PyTorch Geometric (PyTorch-Geometric (PyG)). The core dependencies are organized into the following requirement groups:

- **Base dependencies:** `matplotlib`, `ase` (Atomic Simulation Environment (ASE)), `tqdm`, `tensorboard`, `psutil`, `sympy`, `scikit-learn`, `numpy`, `scipy`, `mpi4py`, and `vesin` (for efficient neighbor-list construction with PBC).
- **PyTorch:** `torch` (v2.8.0+), `torchvision`, `torchaudio`, `e3nn` (for equivariant operations), and `torch-ema`.
- **PyTorch Geometric:** `torch_geometric` (v2.6.1+), `torch_scatter`, `torch_sparse`, `torch_cluster`, and `torch_spline_conv`.
- **Optional:** `pymatgen`, `optuna`, `deephyper`, `deepspeed`, `wandb`, `mendeleev`, and `pandas`.

3.2 INSTALLATION FROM SOURCE

To install HydraGNN from source, clone the repository and use the provided installation script:

Listing 1. Installation from source

```
git clone https://github.com/ORNL/HydraGNN.git
cd HydraGNN
pip install -e .

# Install dependencies
./install_dependencies.sh          # Base + Torch + PyG
./install_dependencies.sh all      # All optional dependencies
./install_dependencies.sh deepspeed # DeepSpeed integration
```

3.3 INSTALLATION ON DOE SUPERCOMPUTERS

HydraGNN provides pre-configured installation scripts for the following US Department of Energy (DOE) leadership-class supercomputers:

- **Frontier** (OLCF, AMD MI250X GPUs): Scripts for ROCm 6.4 and ROCm 7.1
- **Perlmutter** (NERSC, NVIDIA A100 GPUs): CUDA-based installation
- **Andes** (OLCF, NVIDIA V100 GPUs): Includes parallel installation scripts
- **Aurora** (ALCF, Intel Data Center Max GPUs): Intel XPU with CCL backend

These scripts are located in the `installation_DOE_supercomputers/` directory of the HydraGNN repository and handle the specific module loading and environment configuration required on each machine.

3.4 QUICK START

HydraGNN provides a simple Python API for training and prediction:

Listing 2. Quick start example

```
import hydragnn

# Train a model from a JSON configuration file
hydragnn.run_training("config.json")

# Load a trained model and run predictions
hydragnn.run_prediction("config.json")
```

As a concrete example, the following snippet trains a PNA model on the QM9 molecular dataset to predict the free energy at 298 K:

Listing 3. Minimal QM9 example

```
from torch_geometric.datasets import QM9
from hydragnn.preprocess import update_predicted_values
from hydragnn.preprocess import update_atom_features

def qm9_pre_transform(data):
    data.y = data.y[:, 10]  # Free energy at 298 K
    update_predicted_values(1, 1, [1], [1], data)
    update_atom_features(data, ["atomic_number"])
    return data

dataset = QM9(root="dataset/QM9",
              pre_transform=qm9_pre_transform)

hydragnn.run_training("qm9_config.json",
                      dataset=dataset)
```

For complete end-to-end examples, including multi-task training, dataset splitting, evaluation, and the FePt alloy dataset, see Section 15.

Alternatively, a more fine-grained programmatic API is available:

Listing 4. Programmatic API

```
import hydragnn
from hydragnn.utils.model import save_model, load_existing_model
from hydragnn.utils.config_utils import update_config

# Load and parse configuration
config = hydragnn.utils.input_config_parsing.config_utils
  .parse_and_update("config.json")

# Create model and train
model = hydragnn.models.create.create_model_config(config)
hydragnn.train.train_validate_test.train_validate_test(
    model, optimizer, train_loader, val_loader, test_loader,
    writer, config
)
```

```
15
16 # Save and load models
17 save_model(model, optimizer, "my_model", path="./logs")
18 model = load_existing_model(model, "my_model", path="./logs")
```

4. MATHEMATICAL BACKGROUND

4.1 GRAPHS

Graphs are mathematical structures that consist of nodes (vertices) and edges (links) which connect nodes. A graph G is usually represented in mathematical terms as

$$G = (V, E), \tag{1}$$

where V represents the set of nodes and E represents the set of edges between these nodes. An edge $(u, v) \in E$ connects nodes $u, v \in V$. The topology of a graph can be described through the adjacency matrix A, an $N \times N$ matrix where N is the number of nodes in the graph. The entries of A are defined by the rule:

$$A[u, v] = \begin{cases} 1, & \text{if } (u, v) \in E, \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

4.2 GRAPH NEURAL NETWORKS

GNNs (Kipf and Welling 2017) are a class of DL models designed to process and analyze data represented as graphs. GNNs aim to learn representations of nodes and edges in a graph, allowing them to perform tasks such as node classification, link prediction, and graph regression. This makes GNNs well suited for various real-world applications where data can be represented as graphs, such as social networks, atomic structures, knowledge graphs, recommendation systems, and more.

To use GNNs, the graph and its components are associated with the following representations:

- **Node:** Each node u is associated with a d_1-dimensional feature vector $\mathbf{x} \in \mathbb{R}^{d_1}$ containing the embedded nodal properties and also a label vector $\mathbf{y}_{\mathbf{node}} \in \mathbb{R}^{b_1}$ for tasks related to node-level predictions.
- **Edge:** Optionally, each edge e_{ij} can be associated with a d_2-dimensional feature vector $\mathbf{e} \in \mathbb{R}^{d_2}$
- **Graph:** The graph is associated with a label vector $\mathbf{y}_{\mathbf{graph}} \in \mathbb{R}^{b_2}$ for tasks related to graph-level predictions.

Message-Passing Neural Networks (MPNNs) (Gilmer et al. 2017) are a type of GNN that leverage information from neighboring nodes to update the node representations iteratively in message-passing layers. Message-passing layers are also commonly referred to as graph convolutional layers, and MPNNs as graph convolutional neural networks (GCNNs). ***As a result, for the remainder of this manual, "message-passing layer" and "MPNNs" are used interchangeably with "graph convolutional layer" and "GCNNs," respectively.***

This process allows GCNNs to learn complex structural relationships between nodes in the graph for an enhanced representation. Currently, GCNNs are extensively used in chemistry and material science to predict scientifically relevant properties by directly mapping the atomic structure to graphs, with atoms represented as nodes and atomic interactions (e.g., chemical bonds) represented as edges (Lubbers, Smith, and Barros 2018; Batzner et al. 2022; Batatia et al. 2022; Batatia et al. 2024; Gasteiger, Groß, and Günnemann 2022; Fuchs et al. 2020; Merchant et al. 2023; Takamoto, Shinagawa, Motoki, et al. 2022; Zhang, Liu, Zhang, et al. 2024). The complex representations that GCNNs learn can naturally be transferred to predict on structures of different composition, size, and periodicity. For these applications, some example features and prediction tasks are:

- **Node features:** atomic number, atomic mass, isotope
- **Edge features:** bond type, distance, relative position vector
- **Node-level predictions:** nodal potential energy, atomic forces, electronegativity, atomic charge
- **Graph-level predictions:** total potential energy, ultraviolet-visible spectrum, band gap

The general architecture of a GCNN involves the following components:

- **Input Layer:** Nodes in the graph are initially associated with feature vectors. Edges may optionally be associated with feature vectors as well.
- **Message Passing Layers:** In message-passing, information from neighboring nodes is aggregated and combined to update node representations. This process is typically performed iteratively over multiple layers.
- **Global Pooling Layer:** After several iterations of message-passing, a global pooling layer aggregates the final node representations to produce a graph-level representation. This representation can be used for graph-level prediction tasks where the target is a global property of the entire input structure.
- **Fully Connected Layer:** A stack of fully connected (FC) layers can be optionally added at the end of the GCNN architecture to increase the expressivity of the model in predicting both nodal outputs and global outputs.

Since HydraGNN exclusively implements GCNNs, GNN refers to a GCNN for the remainder of this section unless otherwise specified..

4.2.1 Input layer

In a GNN model, the input layer is responsible for encoding the initial node and edge features of the input graph.

For each node $v_i \in V$, the input layer takes its initial feature vector $\mathbf{X}_i^{(0)} \in \mathbb{R}^{d_x^{(0)}}$ and encodes it as follows:

$$\mathbf{X}_i^{(0)} = \text{Encoding}(\text{Node Features of } v_i), \tag{3}$$

where $\mathbf{X}_i^{(0)}$ is the encoded feature vector of node v_i at the input layer, and $d^{(0)}$ represents the dimensionality of the initial node features.

Similarly, for each edge $e_{ij} \in E$ between nodes v_i and v_j, the input layer takes its initial feature vector $\mathbf{E}_{ij}^{(0)} \in \mathbb{R}^{d_e^{(0)}}$ and encodes it as follows:

$$\mathbf{E}_{ij}^{(0)} = \text{Encoding}(\text{Edge Features of } e_{ij}), \tag{4}$$

where $\mathbf{E}_{ij}^{(0)}$ is the encoded feature vector of edge e_{ij} at the input layer, and $d_e^{(0)}$ represents the dimensionality of the initial edge features.

The encoding process can involve various techniques such as embedding layers, feature transformations, or even direct use of raw features, depending on the specific GNN architecture and the characteristics of the input data.

Additional position information may be given for each node $v_i \in V$, denoted by $\mathbf{P}_i^{(0)} \in \mathbb{R}^n$ (where n is the spatial dimension).

After the input layer, the GNN proceeds to perform message creation/aggregation and node update steps in subsequent message-passing layers to propagate information and learn expressive representations throughout the graph.

4.2.2 Message Passing Layer

In a GNN model, a message passing layer is responsible for aggregating and propagating information from neighboring nodes and edges to update the node representations.

At the l-th message-passing layer of the GNN architecture, the message passing proceeds as follows. First, for each node v_i, the aggregated message $\mathbf{M}_i^{(l)}$ is constructed from its neighboring nodes v_j and edges e_{ij}. This can be expressed as:

$$\mathbf{M}_{ij}^{(l)} = \text{Interact}^{(l)}(\mathbf{X}_i^{(l)}, \mathbf{X}_j^{(l)}, \mathbf{E}_{ij}^{(l)}), \tag{5}$$

$$\mathbf{M}_i^{(l)} = \text{Aggregate}^{(l)}\left\{\mathbf{M}_{ij}^{(l)} : j \in \mathcal{N}(i)\right\}, \tag{6}$$

where $\mathbf{X}_i^{(l)}$ and $\mathbf{X}_j^{(l)}$ are the feature vectors of nodes v_i and v_j at layer l, respectively, and $\mathbf{E}_{ij}^{(l)}$ are the features of the edge from node v_j to node v_i. The set $\mathcal{N}(i)$ indicates the neighbors of node v_i. The Interact$(\cdot)$ function (typically a neural network) constructs pairwise messages between a node and each of its neighbors. These pairwise messages are then input to an aggregation operator (e.g., sum, mean, min, or max) to construct a final incoming message from the neighbors.

After constructing the aggregated messages $\mathbf{M}_i^{(l)}$, the node update step combines them into the node representations. This can be expressed as:

$$\mathbf{X}_i^{(l+1)} = \text{Update}(\mathbf{X}_i^{(l)}, \mathbf{M}_i^{(l)}), \tag{7}$$

where $\mathbf{X}_i^{(l+1)}$ is the updated representation of node v_i at the $(l+1)$-th layer, and Update$^{(l)}(\cdot)$ is a function that takes the current node representation $\mathbf{X}_i^{(l)}$ and the aggregated message $\mathbf{M}_i^{(l)}$ as inputs and produces the updated representation.

By iterating the message passing and node update steps for multiple layers, the GNN can propagate information throughout the graph, capturing complex dependencies and structural patterns to learn expressive node representations for various graph-related tasks.

4.2.3 Equivariant Layer

If an equivariant architecture is selected, the message-passing layer incorporates an equivariant update to the nodes' equivariant features[1] (denoted $\mathbf{V}_i^{(l)}$). In equivariant layers, message-passing is split into invariant and equivariant "tracks," where messages flow through each track analogously to the message-passing formulation in Section 4.2.2. This can be formulated as:

Invariant Track:

$$\mathbf{M}_{\text{inv},ij}^{(l)} = \text{Interact}_{\text{inv}}^{(l)}\left(\mathbf{X}_i^{(l)}, \mathbf{X}_j^{(l)}, \mathbf{V}_i^{(l)}, \mathbf{V}_j^{(l)}, \mathbf{E}_{ij}^{(l)}\right) \tag{8}$$

$$\mathbf{M}_{\text{inv},i}^{(l)} = \text{Aggregate}_{\text{inv}}^{(l)}\left\{\mathbf{M}_{\text{inv},ij}^{(l)} : j \in \mathcal{N}(i)\right\} \tag{9}$$

$$\mathbf{X}_i^{(l+1)} = \text{Update}_{\text{inv}}^{(l)}\left(\mathbf{X}_i^{(l)}, \mathbf{M}_{\text{inv},i}^{(l)}\right) \tag{10}$$

Equivariant Track:

$$\mathbf{M}_{\text{eq},ij}^{(l)} = \text{Interact}_{\text{eq}}^{(l)}(\mathbf{X}_i^{(l)}, \mathbf{X}_j^{(l)}, \mathbf{V}_i^{(l)}, \mathbf{V}_j^{(l)}, \mathbf{E}_{ij}^{(l)}) \tag{11}$$

$$\mathbf{M}_{\text{eq},i}^{(l)} = \text{Aggregate}_{\text{eq}}^{(l)}\{\mathbf{M}_{\text{eq},ij}^{(l)} : j \in \mathcal{N}(i)\} \tag{12}$$

$$\mathbf{V}_i^{(l+1)} = \text{Update}_{\text{eq}}^{(l)}\left(\mathbf{V}_i^{(l)}, \mathbf{M}_{\text{eq},i}^{(l)}\right) \tag{13}$$

1. For example, the node positions may be treated as equivariant node features.

Note that this approach requires the node representations $\mathbf{X}_i^{(l)}$ to be invariant and the functions $\text{Interact}_{\text{eq}}(\cdot)$, $\text{Aggregate}_{\text{eq}}(\cdot)$, and $\text{Update}_{\text{eq}}(\cdot)$ to be equivariant.[2]

4.2.4 Global Pooling Layer

In a GNN model, a global pooling layer is used to aggregate information over all nodes in the graph to create a fixed-size graph-level representation that can be used for graph-level tasks. This can be formulated as:

$$\mathbf{Z} = \text{GlobalPooling}(\mathbf{X}^{(L)}), \tag{14}$$

where $\mathbf{X}_i^{(L)}$ is the set of all node representations after L message-passing layers, and $\mathbf{Z}$ is the graph-level representation.

The aggregation used in GlobalPooling$(\cdot)$ can take various forms, such as mean pooling, max pooling, or attention-based pooling, depending on the specific task and requirements. The resulting graph-level representation $\mathbf{Z}$ captures essential information from all nodes in the graph and can be used for tasks like graph classification or regression.

4.2.5 Fully Connected Layer

A FC layer at the end of a GNN model is used to map the node and graph-level representations to the final outputs for the prediction tasks at hand. The FC layer is implemented as a multi-layer perceptron (MLP) which repeatedly applies a linear transformation followed by a non-linear activation function (e.g., ReLU, sigmoid, or softmax), and concludes with one final linear transformation. We denote the predicted nodal outputs and graph-level outputs are $\mathbf{O}^{node}$ and $\mathbf{O}^{graph}$, respectively. The respective outputs can be formulated as:

$$\mathbf{O}_i^{node} = \text{FullyConnected}(\mathbf{X}_i^{(L)}), \tag{15}$$

$$\mathbf{O}^{graph} = \text{FullyConnected}(\mathbf{Z}). \tag{16}$$

where $\mathbf{X}_i^{(L)}$ is the final representation for node v_i after L message-passing layers and $\mathbf{Z}$ is the graph-level embedding after the global-pooling layer. The FC layer allows the GNN to map the node and/or graph-level representations to the appropriate output spaces for various node-level and graph-level predictive tasks.

4.2.6 A Note on Parameter Sharing in GNNs

When a GNN processes a graph, it applies the same input layer, message-passing layers, and fully connected layer to each node in the graph, with each layer being parametrized by a set of learnable weights and biases. Because the GNN shares the same weights and biases across all nodes, it can leverage the learned information from the entire graph to make predictions at the node level. This parameter sharing significantly reduces the number of trainable parameters and enables the GNN to generalize well to new, unseen nodes during inference. Moreover, this parameter sharing mechanism is one of the strengths of GNNs that allows them to handle graphs of varying sizes efficiently.

2. The specific group that objects are invariant/equivariant to depends on the chosen equivariant architecture.

4.3 MULTI-TASK LEARNING (MTL)

MTL (R. Caruana 1993) involves training a single model to perform multiple related tasks simultaneously, rather than training separate models for each task, thereby offering the following benefits:

- *exploiting task relationships:* scientific data often consists of related tasks or subtasks that share some underlying structure or dependencies due to physics correlations. MTL allows one to leverage these task relationships by jointly learning from multiple predictive tasks. The shared representation learned by the GNN model can capture common patterns and knowledge across tasks, leading to improved generalization and performance.
- *regularization and generalization:* MTL acts as a form of regularization by encouraging the model to learn shared representations across tasks. This regularization can help prevent overfitting. The shared representations learned can capture important features and patterns that are common across tasks, leading to improved generalization performance.

The improvement of an MTL model depends on how strongly the quantities to be predicted are mutually correlated in a particular application. This type of field-specific inductive bias has been defined as *knowledge-based*, for which the training of a quantity can benefit from the information contained in the training signal for other quantities. Indeed, other tasks can provide additional domain knowledge that would otherwise be absent if the quantities were trained independently from each other. Therefore, the mutual correlation between quantities is used to treat every single task as an enrichment of the knowledge available to improve the training of other tasks. This mechanism allows for the explicit incorporation of physics knowledge to improve model quality. Ultimately, MTL allows a direct and automated incorporation of physics knowledge into the model by extracting correlations between multiple quantities, with manual intervention by a domain expert only needed in determining which quantities to use.

Each predicted quantity is associated with a separate loss function, and the global objective function minimized during NN training is a linear combination of these individual loss functions. Formally, let T be the total number of physical quantities, or tasks, we want to predict. A single task identified by index i focuses on reconstructing a function $f_i : \mathbb{R}^a \to \mathbb{R}^{b_i}$ defined as

$$\mathbf{y}_i = f_i(\mathbf{x}), \quad i = 1, \ldots, T, \tag{17}$$

where $\mathbf{x} \in \mathbb{R}^a$ and $\mathbf{y}_i \in \mathbb{R}^{b_i}$. The MTL makes use of the correlation between the quantities $\mathbf{y}_i$, where the functions f_i in Equation (17) are replaced by a single function $\hat{f} : \mathbb{R}^a \to \mathbb{R}^{\sum b_i}$ that can model all the relations between inputs and outputs as follows:

$$\begin{bmatrix} \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_T \end{bmatrix} = \hat{f}_{\mathbf{W}_{\mathrm{MTL}}}(\mathbf{x}), \tag{18}$$

where $\mathbf{W}_{\mathrm{MTL}}$ represents the weights to be learned.

The global loss function $\mathcal{L}_{\mathrm{MTL}} : \mathbb{R}^{N_{\mathrm{MTL}}} \to \mathbb{R}^+$ to be minimized in MTL is a linear combination of the loss functions for the single tasks:

$$\mathcal{L}_{\mathrm{MTL}}(\mathbf{W}_{\mathrm{MTL}}) = \sum_{i=1}^{T} \alpha_i \left\| \mathbf{y}_{\mathrm{predict},i} - \mathbf{y}_i \right\|_2^2, \tag{19}$$

where $\mathbf{y}_{\mathrm{predict},i} = \hat{f}_{\mathbf{W}_{\mathrm{MTL}},i}(\mathbf{x})$ is the vector of predictions for the ith quantity of interest and α_i (for $i = 1, \ldots, T$) are the mixing weights for the loss functions associated with each single quantity. The values

of the α_i's in Equation (19) are hyperparameters of the surrogate model and thus can be tuned. In the code, the default setting use equal weights for each property being predicted; however, this definition of the loss function enables one to modify the values of the α_i to purposely favor the training towards one property of interest.

As mentioned above, the multiple quantities in MTL can be interpreted as mutual inductive biases because the error of a single quantity acts as a regularizer with respect to the loss functions of other quantities. In order to clarify why MTL can be seen as a regularizer, let us start with the formulation of a regularized training as a constrained optimization problem:

$$\begin{cases} \arg\min\limits_{\mathbf{w}} \|\mathbf{y}_{\text{predicted}}(\mathbf{w}) - \mathbf{y}\|_2^2 \\ \mathbf{c}(\mathbf{w}) = \mathbf{g}, \end{cases} \tag{20}$$

where $\mathbf{w} \in \mathbb{R}^N$ is the vector of regression coefficients of the model, $\mathbf{y}$ are the target values, $\mathbf{y}_{\text{predicted}}(\mathbf{w})$ are the predictions produced by the DL model, $\mathbf{c}(\mathbf{w}) \in \mathbb{R}^c$ and $\mathbf{g} \in \mathbb{R}^c$ are quantities used to express a constraint. The formulation in Equation (20) imposes the constraint $\mathbf{c}(\mathbf{w}) = \mathbf{g}$ in a strong form. The values of $\mathbf{c}(\mathbf{w})$ depend on the model configuration identified by the parameter vector $\mathbf{w}$, whereas the reference values $\mathbf{g}$ are assumed to be given. The constrained optimization problem in Equation (20) can be reformulated so that the constraint is incorporated in the definition of the objective function itself as follows:

$$\arg\min_{\mathbf{w}} \Big[\|\mathbf{y}_{\text{predicted}}(\mathbf{w}) - \mathbf{y}\|_2^2 + \lambda \, \|\mathbf{c}(\mathbf{w}) - \mathbf{g}\|_* \Big], \tag{21}$$

where $\|\cdot\|_*$ may denote the ℓ^1-norm or ℓ^2-norm as explained below. The objective function to be minimized in Equation (21) interprets the constraint as a penalization term with the penalization multiplier λ. This is a weak formulation of the constraint added to the original objective function. Standard ℓ^1 or ℓ^2 regularizations correspond to choosing $\mathbf{c}(\mathbf{w}) = \mathbf{w}$ and $\mathbf{g} = 0$, and they recast the constraint term in Equation (21) from the strong formulation to the weak formulation using the ℓ^1-norm and the ℓ^2-norm, respectively. ***In MTL,*** $\mathbf{c}(\mathbf{w})$ ***are the predictions of additional target properties whose target values are stored in*** $\mathbf{g}$***, which allows to recast the global objective function used in Equation*** (21) ***as the global loss function for MTL defined in Equation*** (19)***.***

5. DEFINITION OF THE GNN MODEL

The definition of the GNN model is provided in the section `'NeuralNetwork'` in the JSON file. This section is made of subsections that cover the main components that fully characterize the GNN training. The subsections are the following:

- `'Architecture'`: defines the hyperparameters of the GNN architecture
- `'Variables_of_interest'`: defines the input and output features used by the model for supervised learning
- `'training'`: defines the hyperparameters of the optimizer used for training

5.1 DEFINITION OF THE ARCHITECTURE

The subsection `'Architecture'` contains the following parameters:

- `'mpnn_type'`: categorical parameter that defines the type of GCNN layer used for message passing that updates the input features on nodes (and possibly edges) of the graph sample
- `'radius'`: real valued parameter that defines the radius cut-off used to establish the connectivity among nodes of the graph. Additionally, some GCNN layers employ this radius to mask out pairwise messages between nodes separated by a distance greater than the radius cut-off.
- `'max_neighbours'`: integer parameter that provides the upper bound to the degree of a node (maximum number of neighbours) to retain a tuneable degree of sparsity in the graph connectivity
- `'hidden_dim'`: integer parameter that establishes the number of neurons (or channels) in the hidden GCN layers
- `'num_conv_layers'`: integer parameter that establishes the number of hidden GCNN layers
- `'output_heads'`: categorical variable that can be either set to `'graph'` or `'node'` depending on the type of target output that the head of the GCNNs architecture has to predict, namely a graph-level output (one target property associated with the entire graph) or node-level output (the target property is predicted for each node within the graph sample). For graph-level outputs, the dimension of the output must be fixed across all the graph samples.
- `'task_weights'`: list of scalar values α_i that are used as multiplying factors in Equation (19) of subsection 4.3 for the loss function of each predictive task to define the global loss function of MTL

5.2 DEFINITION OF THE VARIABLES OF INTEREST

The subsection `'Variables_of_interest'` is used to provide indexing information of the nodal input features and nodal output features within the attribute `data.x`, and global output features within the attribute `data.y` for each data sample. The indices of the nodal input features must be provided in the key `'input_node_features'`, and the output indices must be provided in the key `'output_index'`. The indexing for `data.x` and the indexing for `data.y` are independent, and both start from 0.

A list of categorical values for the key `'type'` must also be provided to describe the type of output, which can be set either to `'graph'` or to `'node'`.

The list of integers inside `'output_dim'` must be used to specify the number of dimensions for each nodal and global feature.

The key 'denormalize_output' is used to define the Boolean variable to map the predictions back to the original domain during the post-processing. This Boolean variable is used only for specific data loaders described in Section 11.1.1.

The key 'output_names' is optional and is used only during the post-processing to assign titles to plots created for diagnostics of the predictive performance of the model on each target output.

5.3 DEFINITION OF THE TRAINING

The 'training' section is used to characterize the following parameters:

- the key 'Checkpoint' is used to set up a Boolean variable that decides whether checkpoint will be applied or not. If set to true, the model is saved on a pickle file named 'saved_model.pk' inside a log folder which is automatically created at the beginning of each training run.
- 'checkpoint_warmup' is used to set up an integer that provides the number of preliminary epochs to wait before activating the checkpointing. This value is used only if 'Checkpoint' is set to true
- 'num_epoch' is used to set up an integer variable that describes the number of epochs for the training
- 'batch_size' is used to set up an integer variable that describes the size of the local batch size for each process. The effective global batch size can be calculated by multiplying this integer value by the number of processes used to distribute the training with DDP
- 'continue' is used to set a Boolean variable. If set to true, it loads a pre-existing model to start a new training
- 'startfrom' is used to provide the path where the pre-existing model can be found and loaded to start a new training. This parameter is used only if the value associated with the key 'continue' is set to true.
- 'perc_train' is used to set a scalar value between 0.0 and 1.0 that prescribes the fraction of the dataset that must be used for the training. The remaining fraction is equally split between validation and testing.
- 'loss_function_type' is used to set a string value that prescribes the type of loss function to minimize during the training of the neural network
- 'Optimizer' is used to create a subsection of the JSON parsing file that provides hyperparameter values for the training, such as the type of optimizer (e.g, SGD, Adam, AdamW) and the learning rate.
- 'compute_grad_energy': if set to true, the model is assumed to predict nodal energies, which are summed for the total energy prediction. Forces are then predicted by automatically differentiating the total energy prediction with respect to node positions, which enforces energy conservation. In this mode, the data objects must include both energy and forces attributes.

Listing 5. Example of a complete JSON configuration file

```
{
  "Verbosity": {
    "level": 2
  },
  "Dataset": {
    "name": "qm7x",
    "path": {"total": "./dataset/qm7x"},
    "format": "XYZ",
    "rotational_invariance": false,
    "normalize_features": true
  },
  "NeuralNetwork": {
    "Architecture": {
      "mpnn_type": "PNA",
      "edge_features": ["bond_length"],
      "radius": 5.0,
      "max_neighbours": 100,
      "hidden_dim": 200,
      "num_conv_layers": 6,
      "activation_function": "relu",
      "graph_pooling": "mean",
      "periodic_boundary_conditions": false,
      "output_heads": {
        "graph": {
          "num_sharedlayers": 2,
          "dim_sharedlayers": 200,
          "num_headlayers": 2,
          "dim_headlayers": [1000,1000]
        },
        "node": {
          "num_headlayers": 2,
          "dim_headlayers": [1000,1000],
          "type": "mlp"
        }
      },
      "task_weights": [1.0, 1.0, 1.0, 1.0, 1.0]
    },
    "Variables_of_interest": {
      "input_node_features": [0],
      "output_index": [0, 1, 2, 3, 4],
      "type": ["graph", "node", "node", "node", "node"],
      "output_dim": [1, 3, 1, 1, 1],
      "output_names": ["HLGAP", "forces", "hCHG", "hVDIP", "hRAT"],
      "denormalize_output": false
    },
    "training": {
      "Checkpoint": true,
      "num_epoch": 3,
      "batch_size": 32,
      "precision": "fp32",
      "EarlyStopping": true,
      "patience": 50,
```

```
      "continue": 0,
      "startfrom": "existing_model",
      "loss_function_type": "mse",
      "conv_checkpointing": false,
      "Optimizer": {
        "type": "AdamW",
        "learning_rate": 0.001
      }
    }
  }
}
```

5.4 VERBOSITY

The key `'verbosity'` is used to tune the amount of information included in the output of print statements. A higher degree of verbosity may provide meaningful information in debugging and logging. Adjusting the verbosity level can help developers diagnose issues, track the flow of a program, or provide varying levels of detail in log files. The output is redirected to the output channel used through the `Python` package `log`.

The utilities to print outputs based on the level of verbosity are implemented in the file:

`HydraGNN/hydragnn/utils/print_utils.py`

The value of verbosity can be set to any integer between 0 and 4. The meaning of each verbosity level is the following:

- `'level: 0'`: nothing is printed on the screen
- `'level: 1'`: only the process with rank 0 prints output at the end of each training epoch
- `'level: 2'`: only the process with rank 0 prints output at each batched gradient update, showing the stage of the training on each epoch using a progression bar
- `'level: 3'`: every process prints output at the end of each training epoch
- `'level: 4'`: every process prints output at each batched gradient update, showing the stage of the training on each epoch using a progression bar

6. EDGE FEATURES AND RADIAL TRANSFORMS

Edge features encode the geometric relationships between connected nodes (atoms) in a graph. They are critical for models that must distinguish interactions at different distances and orientations. HydraGNN supports several types of edge features, configurable via the `Architecture` section of the JSON file.

The core source files for edge feature processing are:

- `hydragnn/preprocess/graph_samples_checks_and_updates.py`: edge construction transforms (`Distance`, `PBCDistance`, `PBCLocalCartesian`)
- `hydragnn/preprocess/serialized_dataset_loader.py`: orchestrates edge computation during dataset loading
- `hydragnn/utils/input_config_parsing/config_utils.py`: function `update_config_edge_dim()` that automatically sets `edge_dim` from the `edge_features` list

6.1 EDGE FEATURE TYPES

The following edge feature transforms are available:

- **Distance:** The Euclidean distance between connected nodes, $d_{ij} = \|\mathbf{r}_i - \mathbf{r}_j\|_2$. This is the default and most commonly used edge feature. Computed by the `Distance` transform from PyTorch Geometric (or `PBCDistance` when periodic boundary conditions are enabled). Distances are automatically normalized by the maximum edge length across the entire dataset.
- **Spherical coordinates:** The relative position vector expressed in spherical coordinates (r, θ, ϕ), providing directional information. Activated when `spherical_coordinates` is set in the `Descriptors` section.
- **Point-pair features:** Additional geometric descriptors including relative angles between surface normals, useful for capturing local geometry. Activated when `point_pair_features` is set in the `Descriptors` section.
- **Local Cartesian coordinates:** The relative position in a local reference frame centered on the target node. The `PBCLocalCartesian` variant correctly accounts for periodic boundary shifts.

To specify edge features, use the `edge_features` list in the `Architecture` section. The length of this list automatically determines the `edge_dim` parameter:

Listing 6. Configuring edge features in the JSON file

```
"Architecture": {
    "edge_features": ["length"],
    "max_neighbours": 100,
    "radius": 7.0
}
```

Edge features are supported by the following models: GAT, PNA, PNAPlus, PAINN, PNAEq, CGCNN, SchNet, EGNN, DimeNet, and MACE. Note that edge features cannot be used together with interatomic potentials (`enable_interatomic_potential`), as MLIP models build their own specialized edge features for force computation.

6.2 RADIAL BASIS FUNCTIONS

Several message-passing policies (PNAPlus, PAINN, MACE, DimeNet) use radial basis function (RBF) expansions to transform scalar distances into high-dimensional feature vectors that the neural network can process more effectively.

6.2.1 Supported Radial Basis Types

- **Bessel basis** (`radial_type: bessel`): Uses sinusoidal Bessel functions of the first kind, providing a smooth, orthogonal basis set:

$$\phi_n(r) = \sqrt{\frac{2}{r_{\max}}} \cdot \frac{\sin(n\pi\, r/r_{\max})}{r}, \qquad n = 1, \ldots, N_{\text{radial}}. \tag{22}$$

  Implemented in the class `BesselBasis` in `hydragnn/utils/model/mace_utils/modules/radial.py`.

- **Gaussian smearing** (`radial_type: gaussian`): Expands distances using Gaussian kernels centered at evenly spaced values:

$$\phi_i(r) = \exp\!\Big(-\frac{1}{2\sigma^2}\,(r-\mu_i)^2\Big), \qquad \mu_i \text{ linearly spaced from 0 to } r_{\max}. \tag{23}$$

  The width $\sigma = r_{\max}/(N_{\text{basis}} - 1)$. Implemented as class `GaussianBasis`.

- **Chebyshev polynomials** (`radial_type: chebyshev`): Uses Chebyshev polynomials of the first kind $T_n(x)$ for the expansion. Implemented as class `ChebychevBasis`.

For PAINN specifically, the radial expansion uses a sinc-based expansion implemented in the `sinc_expansion()` function in `hydragnn/models/PAINNStack.py`, combined with a Behler–Parinello cosine cutoff:

$$f_{\text{cut}}(r) = \begin{cases} \frac{1}{2}(\cos(\pi\, r/r_{\text{cut}}) + 1), & r < r_{\text{cut}}, \\ 0, & r \geq r_{\text{cut}}. \end{cases} \tag{24}$$

6.2.2 Distance Transforms

Distance transforms remap raw interatomic distances before they are passed to the radial basis, using element-pair-specific scaling derived from covalent radii. These are available for the MACE model via the `distance_transform` key:

- **Agnesi transform** (`distance_transform: Agnesi`): Applies the Agnesi witch function with element-dependent reference distances derived from ASE covalent radii: $r_0 = (r_{\text{cov}}[Z_u] + r_{\text{cov}}[Z_v])/2$. Implemented in class `AgnesiTransform` with tunable parameters q, p, a.
- **Soft transform** (`distance_transform: Soft`): Uses a tanh-based clipping function for smooth distance reweighting with reference $r_0 = (r_{\text{cov}}[Z_u] + r_{\text{cov}}[Z_v])/4$. Implemented in class `SoftTransform` with parameters a and b.

6.2.3 Polynomial Cutoff Envelope

To ensure that the radial features decay smoothly to zero at the cutoff radius, a polynomial envelope function is applied for certain architectures:

$$p(r) = 1 - \frac{(p+1)(p+2)}{2}\left(\frac{r}{r_{\max}}\right)^p + p(p+2)\left(\frac{r}{r_{\max}}\right)^{p+1} - \frac{p(p+1)}{2}\left(\frac{r}{r_{\max}}\right)^{p+2}, \tag{25}$$

where p is the `envelope_exponent` (default: 6). Implemented in class `PolynomialCutoff`.

6.2.4 Configuration Reference

Listing 7 shows how to configure radial basis functions and distance transforms in the JSON file.

Listing 7. JSON configuration for radial basis and distance transforms (MACE example)

```
"Architecture": {
    "mpnn_type": "MACE",
    "radius": 5.0,
    "num_radial": 8,
    "radial_type": "bessel",
    "distance_transform": "Agnesi",
    "envelope_exponent": 5,
    "max_ell": 2,
    "node_max_ell": 1,
    "correlation": [2, 2, 2]
}
```

Table 1 summarizes which radial parameters are relevant to each model. Only the five MPNN architectures that expose explicit radial basis configuration keys are listed; the remaining models (GIN, GAT, MFC, PNA, CGCNN, EGNN, SchNet-based variants, etc.) use raw interatomic distances directly and do not require radial basis settings.

Table 1. Radial basis configuration keys by model type. Columns: (R) `num_radial`, (T) `radial_type`, (D) `distance_transform`, (G) `num_gaussians`, (F) `num_filters`, (E) `envelope_exponent`.

Model	R	T	D	G	F	E
MACE	✓	✓	✓			✓
PAINN	✓					
DimeNet	✓					✓
PNAPlus	✓					✓
SchNet				✓	✓	

7. PERIODIC BOUNDARY CONDITIONS

For crystalline materials and periodic systems, HydraGNN supports PBC to correctly construct graphs where atoms interact across unit cell boundaries. PBC is enabled via the `periodic_boundary_conditions` flag in the `Architecture` section.

7.1 CORE IMPLEMENTATION

The PBC functionality is implemented in the following source files:

- `hydragnn/preprocess/allowbreak graph_samples_checks_and_updates.py`: contains the `RadiusGraphPBC` class and PBC-aware edge transforms
- `hydragnn/utils/model/operations.py`: contains `get_edge_vectors_and_lengths()`, which computes PBC-corrected edge vectors used by equivariant models
- `hydragnn/preprocess/serialized_dataset_loader.py`: orchestrates PBC-aware edge computation during dataset loading

The class `RadiusGraphPBC` (extending PyTorch Geometric's `RadiusGraph`) uses the `vesin` library to efficiently find neighbors across periodic images. It is instantiated via the helper function:

Listing 8. Creating a PBC-aware radius graph

```
from hydragnn.preprocess import get_radius_graph_pbc

compute_edges = get_radius_graph_pbc(
    radius=5.0,         # Cutoff radius
    loop=False,          # No self-loops
    max_neighbours=100  # Max neighbors per node
)
data = compute_edges(data)
```

Internally, `RadiusGraphPBC` calls `vesin.ase_neighbor_list("ijS", ...)` to obtain source indices, destination indices, and integer cell shift vectors. These shifts are converted to real-space displacement vectors and stored as `data.edge_shifts`:

$$\Delta \mathbf{r}_{ij} = \mathbf{S}_{ij} \cdot \mathbf{C}, \tag{26}$$

where $\mathbf{S}_{ij}$ is the integer shift vector and $\mathbf{C}$ is the 3×3 cell matrix.

The class also provides safeguards:

- **Mixed PBC:** When only some dimensions are periodic (e.g., a slab with `pbc = [True, True, False]`), non-periodic cell vectors are expanded to prevent spurious neighbor detection.
- **Neighbor limiting:** A vectorized routine caps the number of neighbors per node to `max_num_neighbors`, keeping the closest ones.
- **Connectivity guarantee:** Every node is guaranteed to have at least one incoming edge by adding the nearest neighbor if necessary.
- **Self-loop removal:** True self-loops (same atom, zero shift) are removed; only periodic images of the same atom are kept.

7.2 PBC-AWARE EDGE TRANSFORMS

When PBC is active, standard edge feature transforms are replaced by PBC-aware versions that incorporate `edge_shifts`:

- `PBCDistance`: Computes interatomic distances as $d_{ij} = \|\mathbf{r}_j - \mathbf{r}_i + \Delta\mathbf{r}_{ij}\|_2$, stored in `data.edge_attr`.
- `PBCLocalCartesian`: Computes local Cartesian coordinates with PBC-aware displacements.

7.3 DATA REQUIREMENTS

Each data object must contain the following attributes for PBC to work:

- `data.pos`: Node (atom) positions, a tensor of shape $(N, 3)$.
- `data.cell`: Lattice vectors in row form, a tensor of shape $(3, 3)$.
- `data.pbc`: Boolean flags indicating periodicity along each dimension, a tensor of shape $(3,)$ (e.g., `[True, True, True]` for a bulk crystal).

7.4 CONFIGURATION

Listing 9 shows how to enable PBC in the JSON configuration file.

Listing 9. Enabling periodic boundary conditions in the JSON file

```
"Architecture": {
    "periodic_boundary_conditions": true,
    "radius": 5.0,
    "max_neighbours": 100000
}
```

Models that use interatomic distances (PAINN, MACE, SchNet, DimeNet, EGNN) automatically account for PBC through the function `get_edge_vectors_and_lengths()` in `hydragnn/utils/model/operations.py`, which adds `edge_shifts` to the displacement vectors before computing lengths. If `edge_shifts` is not present (non-periodic systems), it defaults to zero.

8. HYPERPARAMETER OPTIMIZATION

HydraGNN integrates with two hyperparameter optimization (HPO) frameworks to automate the search for optimal model configurations. The relevant source files and examples are:

- `hydragnn/utils/hpo/deephyper.py`: DeepHyper integration utilities (node list parsing, launch command generation)
- `examples/qm9_hpo/qm9_optuna.py`: complete Optuna example on QM9
- `examples/multidataset_hpo/gfm_deephyper_multi.py`: DeepHyper example for multi-dataset HPO at scale

8.1 COMMONLY OPTIMIZED HYPERPARAMETERS

The following hyperparameters are most commonly searched during HPO, organized by JSON section:

Table 2. Commonly optimized hyperparameters and their typical search ranges.

JSON Key	Type	Typical Range	Section
`mpnn_type`	categorical	`[PNA, EGNN, SchNet, ...]`	Architecture
`hidden_dim`	integer	50–2000	Architecture
`num_conv_layers`	integer	1–6	Architecture
`num_headlayers`	integer	1–3	output_heads
`dim_headlayers`	integer list	50–1000 per layer	output_heads
`learning_rate`	float (log)	10^{-5}–10^{-2}	Training
`batch_size`	integer	16–256	Training
`radius`	float	3.0–10.0	Architecture
`num_radial`	integer	4–16	Architecture

8.2 OPTUNA

Optuna is a flexible hyperparameter optimization framework that supports various sampling strategies. HydraGNN provides a complete example in `examples/qm9_hpo/qm9_optuna.py`.

The workflow consists of:

1. Define an `objective(trial)` function that receives an Optuna `Trial` object.
2. Use `trial.suggest_*` methods to sample hyperparameters from the search space.
3. Update the HydraGNN JSON configuration dictionary with the sampled values.
4. Call `hydragnn.utils.update_config()` to validate the configuration and populate derived fields.
5. Create the model, train it, and return the validation loss.
6. Create a `Study` and call `study.optimize(objective, n_trials=...)`.

Listing 10 shows a minimal Optuna objective function.

Listing 10. Optuna objective function for HydraGNN HPO

```python
import optuna
import hydragnn

def objective(trial):
    # Sample hyperparameters
    model_type = trial.suggest_categorical(
        "model_type", ["EGNN", "PNA", "SchNet"])
    hidden_dim = trial.suggest_int("hidden_dim", 50, 300)
    num_conv_layers = trial.suggest_int(
        "num_conv_layers", 1, 5)
    num_headlayers = trial.suggest_int(
        "num_headlayers", 1, 3)
    dim_headlayers = [
        trial.suggest_int(f"dim_headlayer_{i}", 50, 300)
        for i in range(num_headlayers)
    ]

    # Update config with sampled values
    arch = config["NeuralNetwork"]["Architecture"]
    arch["model_type"] = model_type
    arch["hidden_dim"] = hidden_dim
    arch["num_conv_layers"] = num_conv_layers
    heads = arch["output_heads"]["graph"]
    heads["num_headlayers"] = num_headlayers
    heads["dim_headlayers"] = dim_headlayers

    # Build model, train, and evaluate
    config_updated = hydragnn.utils.update_config(
        config, train_loader, val_loader, test_loader)
    model = hydragnn.models.create_model_config(
        config=config_updated["NeuralNetwork"])
    model = hydragnn.utils.get_distributed_model(model)
    # ... train model ...
    val_loss, _ = hydragnn.train.validate(
        val_loader, model, verbosity)
    return val_loss.cpu().detach().numpy()

# Run the search
sampler = optuna.samplers.TPESampler()
study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=100)
print("Best:", study.best_params)
```

Optuna supports multiple sampling strategies that can be passed via the `sampler` argument:

- `TPESampler`: Tree-structured Parzen Estimator (default, recommended)
- `RandomSampler`: Uniform random search (good baseline)
- `CmaEsSampler`: CMA-ES evolutionary strategy
- `GridSampler`: Exhaustive grid search
- `NSGAIISampler`: Multi-objective optimization

8.3 DEEPHYPER

DeepHyper (Balaprakash et al. 2018) is a scalable hyperparameter search framework designed for high-performance computing (HPC) environments. HydraGNN provides a DeepHyper integration module in `hydragnn/utils/hpo/deephyper.py` with utilities for:

- `read_node_list()`: parses SLURM node lists for distributing trials across nodes
- `create_ds_config()`: generates DeepSpeed configurations for each trial
- `create_launch_command()`: builds `srun` commands for launching distributed trials

A complete example is provided in `examples/multidataset_hpo/gfm_deephyper_multi.py`. The workflow is:

1. Define the search space using `HpProblem.add_hyperparameter()`.
2. Define a `run(trial)` function that launches a subprocess with the sampled hyperparameters, trains the model, and parses the validation loss from the output.
3. Create a `ProcessPoolEvaluator` with a node queue for distributing trials across compute nodes.
4. Run Bayesian search using `CBO` (Centralized Bayesian Optimization).

Listing 11 shows the search space definition and search invocation.

Listing 11. DeepHyper search space and search setup for HPC

```
from deephyper.problem import HpProblem
from deephyper.search.hps import CBO
from deephyper.evaluator import queued, ProcessPoolEvaluator
from hydragnn.utils.deephyper import read_node_list

problem = HpProblem()
problem.add_hyperparameter(
    (2, 6), "num_conv_layers")
problem.add_hyperparameter(
    (100, 2000), "hidden_dim")
problem.add_hyperparameter(
    (2, 3), "num_headlayers")
problem.add_hyperparameter(
    (300, 1000), "dim_headlayers")
problem.add_hyperparameter(
    ["EGNN", "SchNet", "PNA"], "model_type")

# Distribute trials across SLURM nodes
queue, _ = read_node_list()
evaluator = queued(ProcessPoolEvaluator)(
    run,
    num_workers=NUM_CONCURRENT_TRIALS,
    queue=queue,
    queue_pop_per_task=NNODES_PER_TRIAL,
)

search = CBO(
    problem, evaluator,
    acq_func="UCB",
    multi_point_strategy="cl_min",
```

```
    random_state=42,
    log_dir="hpo_results",
)
results = search.search(max_evals=200)
```

DeepHyper is particularly well-suited for large-scale HPO on DOE supercomputers, as each trial can be launched as a separate multi-node `srun` job using the node queue mechanism. This enables concurrent evaluation of multiple model configurations across hundreds of GPUs.

8.4 CHOOSING BETWEEN OPTUNA AND DEEPHYPER

The two frameworks serve complementary roles. Table 3 summarizes the key differences.

Table 3. Comparison of Optuna and DeepHyper for HydraGNN HPO.

Criterion	Optuna	DeepHyper
Primary use case	Sequential or small-parallel search on a single node (workstation, single GPU node)	Massively parallel search across many nodes on HPC clusters
Parallelism model	In-process threading or optional database-backed parallelism (SQLite, MySQL, PostgreSQL)	Process-pool evaluator with SLURM node queue; each trial launched as a separate `srun` job across multiple nodes
Search algorithm	TPE (default), CMA-ES, random, grid, NSGA-II for multi-objective	Centralized Bayesian Optimization (CBO) with configurable acquisition functions (UCB, EI) and multi-point strategies
Setup complexity	Minimal: `pip install optuna`; no scheduler integration required	Requires SLURM (or PBS) environment; HydraGNN provides helper utilities for node list parsing and launch commands
Trial coordination	Sequential by default; parallelism requires a shared storage backend	Native distributed coordination via `ProcessPoolEvaluator` and node queues
Multi-node trials	Not natively supported; each trial runs on the resources of the calling process	First-class support: `queue_pop_per_task` allocates multiple nodes per trial for data-parallel training
Fault tolerance	Trials pruned or failed are logged; study can be resumed from database	Failed trials return `"F"` objective; search continues with remaining node budget
Result storage	In-memory or relational database; export to CSV/DataFrame	CSV log in `log_dir`; integrates with DeepHyper analytics
Recommended scale	1–8 GPUs, up to ~100 trials	Tens to thousands of GPUs, hundreds of concurrent trials

8.4.0.1 When to use Optuna.

Optuna is the recommended starting point for most users. It requires no HPC scheduler, installs with a single `pip` command, and provides immediate feedback through its built-in visualization module (`optuna.visualization`). Use Optuna when:

- You are developing or prototyping on a workstation or a single compute node.
- The training time per trial is short enough that sequential evaluation is acceptable (minutes to a few hours per trial).
- You want to quickly narrow down the search space before committing to a large-scale run.
- You need multi-objective optimization (e.g., minimizing both validation loss and model size) via `NSGAIISampler`.

8.4.0.2 When to use DeepHyper.

DeepHyper becomes essential when the scale of the search exceeds what a single node can handle. Use DeepHyper when:

- Each trial requires multi-node distributed training (e.g., large models or datasets that do not fit on a single GPU).
- You have access to an HPC allocation and want to evaluate many configurations concurrently to reduce wall-clock time.
- You need to search over expensive architectures where each trial takes hours to days.
- You are running on DOE leadership-class facilities (Frontier, Summit, Perlmutter) where SLURM-native job launching is required.

8.4.0.3 Combined workflow.

A practical strategy is to use both frameworks in sequence: first run a quick Optuna search (50–100 trials) on a single node to identify promising regions of the hyperparameter space, then launch a focused DeepHyper search at scale to fine-tune within those regions. This two-stage approach reduces the computational cost of large-scale HPO while ensuring broad exploration of the search space.

9. MIXED-PRECISION TRAINING

HydraGNN supports three precision levels via the `precision` key in the `Training` section of the JSON configuration. Choosing the right precision mode balances numerical accuracy (Higham 2002), memory usage, and throughput (Micikevicius et al. 2018). The core implementation resides in `hydragnn/train/train_validate_test.py` (functions `resolve_precision()` and `get_autocast_and_scaler()`) and `hydragnn/run_training.py`.

9.1 SUPPORTED PRECISION MODES

- **`fp32`** (default): Single precision (32-bit). No autocasting is applied; all model parameters and data remain at `torch.float32`. This is the safest and most portable option.
- **`fp64`**: Double precision (64-bit). The global default dtype is set to `torch.float64` via `torch.set_default_dtype()`, model weights are created at double precision, and all floating-point input tensors are cast to `float64` before each forward pass. Use this when numerical precision is critical (e.g., energy and force regression on sensitive potential energy surfaces).
- **`bf16`**: Brain floating point (16-bit). Uses PyTorch's `torch.autocast` context manager to perform *mixed-precision* training: the forward pass and loss computation run in `bfloat16`, while parameter updates remain in `float32`. This reduces GPU memory usage and can significantly accelerate training on supported hardware. Unlike `fp16`, `bf16` does not require gradient scaling because it retains the same exponent range as `fp32`.

The JSON configuration accepts several aliases: `"bfloat16"` for `bf16`, `"float32"` or `"float"` for `fp32`, and `"float64"` or `"double"` for `fp64`. Note that `fp16` (IEEE half precision) is **not supported** and will raise a `ValueError`.

Listing 12. Setting precision in the JSON configuration

```
"Training": {
    "precision": "bf16"
}
```

9.2 HARDWARE REQUIREMENTS FOR BF16

The `bf16` mode automatically checks for hardware support at runtime and falls back to full precision with a warning if the device does not support it:

- **NVIDIA GPUs:** Compute capability ≥ 7.0 (Volta V100 and newer). Ampere (A100) and Hopper (H100) architectures provide dedicated `bf16` Tensor Cores for optimal throughput.
- **Intel XPUs:** Supported when `torch.xpu.is_available()` returns `True`.
- **CPUs:** Supported only if the CPU has native `bf16` instructions (`torch.backends.cpu.has_bf16`).

9.3 INTERACTION WITH DISTRIBUTED FRAMEWORKS

9.3.0.1 DeepSpeed.

When `bf16` is enabled with DeepSpeed, HydraGNN uses PyTorch's native `torch.autocast` rather than DeepSpeed's built-in bf16 engine. The DeepSpeed configuration is set as follows:

Listing 13. DeepSpeed bf16 configuration (generated automatically)

```
{
    "bf16": {"enabled": false},
    "torch_autocast": {
        "enabled": true,
        "dtype": "bfloat16"
    }
}
```

This ensures consistent behavior with the non-DeepSpeed code path.

9.3.0.2 FSDP2.

All three precision modes are supported with PyTorch's Fully Sharded Data Parallel (FSDP2). When using FSDP2 with force computation (`compute_grad_energy: true`), HydraGNN applies a workaround that temporarily disables resharding after backward to avoid gradient issues. This is handled automatically and requires no user configuration.

9.4 PRACTICAL GUIDANCE

- **Start with `fp32`** for initial model development and debugging.
- **Switch to `bf16`** when training large models or datasets to reduce memory usage (roughly 2× reduction) and accelerate training. Monitor validation loss to ensure convergence is comparable to `fp32`.
- **Use `fp64`** when the application demands high numerical fidelity, such as when predicting small energy differences between configurations. Be aware that `fp64` doubles memory usage and may reduce throughput by 2–4× compared to `fp32`.

10. EARLY STOPPING AND CHECKPOINTING

HydraGNN provides three complementary mechanisms for managing training duration, model persistence, and memory efficiency: early stopping, model checkpointing, and gradient checkpointing. The core implementations reside in `hydragnn/utils/model/model.py` (classes `EarlyStopping` and `Checkpoint`, functions `save_model()` and `load_existing_model()`) and `hydragnn/models/Base.py` (gradient checkpointing).

Table 4 lists all related JSON configuration keys.

Table 4. JSON configuration keys for early stopping and checkpointing (all under `NeuralNetwork.Training`).

Key	Type	Default	Description
`EarlyStopping`	bool	`false`	Enable early stopping
`patience`	int	`10`	Epochs without improvement before stop
`Checkpoint`	bool	`false`	Enable best-model checkpointing
`checkpoint_warmup`	int	`0`	Epochs to skip before first checkpoint
`conv_checkpointing`	bool	`false`	Gradient checkpointing for conv layers
`continue`	int	`0`	Load a saved model before training
`startfrom`	string	—	Log name of the model to resume from

10.1 EARLY STOPPING

Early stopping monitors the validation loss after each epoch and terminates training when the model has stopped improving, thereby preventing overfitting and saving compute time.

The `EarlyStopping` class maintains a counter that increments each time the validation loss fails to decrease below the previous best. When this counter reaches the `patience` threshold, training stops:

1. After each epoch, the validation loss is reduced across all distributed ranks.
2. If $L_{\text{val}} > L_{\text{val,min}}$ (no improvement), the counter increments.
3. If the counter reaches `patience` consecutive epochs, the training loop exits.
4. If $L_{\text{val}} \leq L_{\text{val,min}}$ (improvement), the counter resets to zero and $L_{\text{val,min}}$ is updated.

Listing 14. Enabling early stopping in the JSON configuration

```
"Training": {
    "EarlyStopping": true,
    "patience": 20,
    "num_epoch": 500
}
```

A good practice is to set `num_epoch` to a large value (e.g., 500) and let early stopping determine the actual training duration. The `patience` value should be proportional to the expected convergence rate; typical values range from 10 (fast convergence, small models) to 50 (slow convergence, large models or noisy loss landscapes).

10.2 MODEL CHECKPOINTING

When enabled, checkpointing saves the model and optimizer state to disk each time the validation loss improves. This guarantees that the best-performing model is preserved even if training continues past the optimal point or is interrupted.

10.2.0.1 What is saved.

Each checkpoint is a `.pk` file (PyTorch pickle via `torch.save`) containing:

- `model_state_dict`: all model parameters
- `optimizer_state_dict`: optimizer state (learning rates, momentum buffers, etc.)

Only rank 0 writes to disk. For FSDP, the full state dictionary is gathered on rank 0 before saving. For DeepSpeed, the native `model.save_checkpoint()` method is used instead.

10.2.0.2 Checkpoint warmup.

The `checkpoint_warmup` parameter skips checkpointing during the first N epochs. This is useful when the model undergoes large, noisy loss changes early in training and you want to avoid saving many short-lived checkpoints.

10.2.0.3 Log directory structure.

All outputs are saved under `./logs/<log_name>/`, where `<log_name>` is automatically generated from the model configuration (model type, radius, hidden dimension, number of epochs, etc.). The directory contains:

- `<log_name>.pk`: the best (or latest) model checkpoint
- `<log_name>_epoch_<N>.pk`: per-epoch checkpoints (a symlink `<log_name>.pk` points to the latest)
- `config.json`: a copy of the full JSON configuration
- `run.log`: training log with per-epoch loss values
- TensorBoard event files (`events.out.tfevents.*`)
- Scatter plots and loss history (`*.png`, `history_loss.pckl`)

Listing 15. Enabling checkpointing with warmup

```
"Training": {
    "Checkpoint": true,
    "checkpoint_warmup": 10,
    "EarlyStopping": true,
    "patience": 20
}
```

10.2.0.4 Resuming from a checkpoint.

To continue training from a previously saved model, set `"continue": 1` and `"startfrom"` to the log name:

Listing 16. Resuming training from a saved checkpoint

```
"Training": {
    "continue": 1,
    "startfrom": "PNA-r-5.0-ncl-3-hd-128-ne-200-lr-0.001",
    "epoch_start": 100
}
```

The function `load_existing_model()` restores both the model weights and optimizer state, handling DDP prefix compatibility and FSDP state dictionaries automatically.

10.3 GRADIENT CHECKPOINTING

Gradient checkpointing (`conv_checkpointing`) is a memory optimization technique that trades compute time for reduced GPU memory usage. Instead of storing all intermediate activations during the forward pass, it discards them and recomputes them on the fly during the backward pass using `torch.utils.checkpoint.checkpoint`.

In HydraGNN, this is applied to the message-passing convolution layers, which are typically the most memory-intensive part of the model. When enabled, each convolution layer's forward call is wrapped in a checkpointing context:

Listing 17. Gradient checkpointing in the forward pass (simplified from Base.py)

```
from torch.utils.checkpoint import checkpoint

for conv, feat_layer in zip(self.graph_convs,
                            self.feature_layers):
    if not self.conv_checkpointing:
        out = conv(x, edge_index, edge_attr)
    else:
        out = checkpoint(conv, use_reentrant=False,
                         x=x, edge_index=edge_index,
                         edge_attr=edge_attr)
```

10.3.0.1 When to use gradient checkpointing.

Enable `conv_checkpointing` when GPU memory is the bottleneck, for example, when training deep models (many convolution layers), using large graphs, or running with limited GPU memory. The memory savings scale with the number of convolution layers, at the cost of approximately 20–30% increased training time due to the additional forward recomputation during the backward pass.

11. MODULES

11.1 THE 'HYDRAGNN' MODULE

The 'hydragnn' module is composed of the following sub-modules:

- 'preprocess': contains capabilities to read data from files and generate 'torch_geometric.data' objects, as well as rescale input and output features
- 'postprocess': contains capabilities to apply rescaling transformations on the output to map the predictions back to the original domain, and generate diagnostic visualizations
- 'models': contains the list of MPNN layers that implements different message passing policies and the multi-headed architecture
- 'globalAtt': contains the implementation of GraphGPS for combining local message passing with global self-attention
- 'train': contains the capabilities to perform distributed training using DDP, DeepSpeed, and FSDP
- 'utils': contains auxiliary capabilities including dataset classes, configuration parsing, distributed computing utilities, profiling, and model management

11.1.1 The 'preprocess' sub-module

The 'preprocess' sub-module contains functionalities that (i) perform pre-processing transformations on the data samples before feeding them to the HydraGNN model for training, (ii) generate serialized dataloaders for specific formatting of the data, and (iii) construct radius-based graphs with optional periodic boundary conditions.

The transformations are contained in the file 'graph_samples_checks_and_updates.py' and include:

- **Radius graph construction:** RadiusGraph for standard graphs and RadiusGraphPBC for periodic systems
- **Edge feature computation:** Distance, SphericalCoordinates, PointPairFeatures, and LocalCartesian transforms
- **Positional encodings:** AddLaplacianEigenvectorPE for GraphGPS global attention
- **Rotational invariance:** NormalizeRotation transform to remove rotational degrees of freedom
- **Feature normalization:** Min-max normalization of node and graph features
- **Energy linear regression:** Per-element linear energy correction for improved energy prediction

The serialized data loaders for specific data formats are included in the files:

- 'serialized_dataset_loader.py': main data loading pipeline with the function dataset_loading_and_splitting() as the entry point
- 'load_data.py': provides HydraDataLoader, a custom multi-threaded DataLoader optimized for HPC environments with CPU affinity support
- 'stratified_sampling.py': subsampling by category for balanced training

11.1.2 The 'models' sub-module

The 'models' sub-module provides the set of MPNN layers that can be used to perform message passing in the HydraGNN architecture. The MPNN layers are organized according to an object-oriented framework to attain:

- Abstraction. Complex implementation details are hidden and only necessary features of an object are exposed. This simplifies the interface for using the object, making it easier to work with.
- Inheritance. New MPNN layer classes can be created by inheriting properties and behaviors from existing classes. This promotes code reuse and helps establish a hierarchical relationship between classes.
- Reusability. Each MPNN class can be reused in different parts of an application or even in different applications altogether. This reusability can save development time and reduce the likelihood of errors.

Figure 1 details the object-oriented programming structure utilized by each of the MPNN models in HydraGNN. The inheritance of the Base class for each MPNN enables the seamless change between message passing policies. It also enables a direct comparison between MPNNs in a unified framework of the Base class. This comprehensive setup makes HydraGNN a distinct GCNNs architecture that is able to seamlessly switch between different MPNN layers, thereby allowing them to be treated as hyperparameters.

The 'Base' class (implemented in 'Base.py') defines the encoder-decoder architecture:

- **Encoder:** Embedding layer → N convolutional layers (with BatchNorm and activation) → node feature representations
- **Decoder:** Multi-headed output with separate paths for graph-level predictions (via global pooling and shared/head-specific FC layers) and node-level predictions (via head-specific FC layers)

The 'Base' class provides the following core capabilities that are inherited by all MPNN implementations:

- Graph pooling layer construction and forward pass logic
- Multi-headed output layer construction with configurable shared and task-specific layers
- Loss computation with configurable loss functions and task weights
- Support for both hyperparameter-based and NLL-based loss weighting
- Optional variance output for uncertainty quantification

The choice of the MPNN layer used is specified by the 'mpnn_type' key inside the 'Architecture' section of the JSON file. The 13 supported models are: GIN, PNA, PNAPlus, GAT, MFC, CGCNN, SAGE, SchNet, DimeNet, EGNN, PNAEq, PAINN, and MACE.

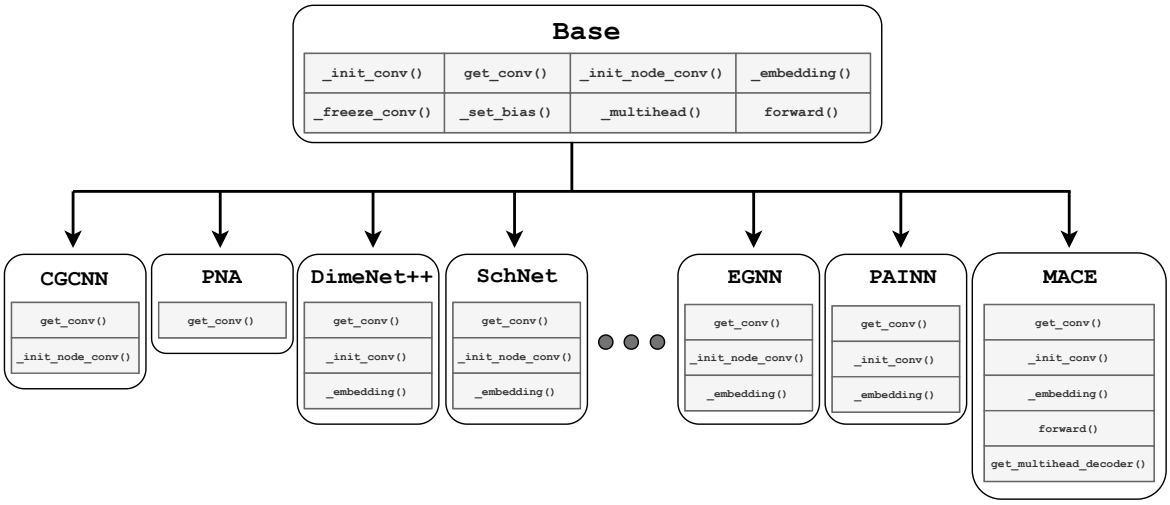

Figure 1. The object oriented programming structure of HydraGNN, in which each class inherits all of the functions from the `Base` class, and over-rides the selected functions that allow for the full functionality of the MPNN. The GAT, MFC, GIN, SAGE, PNAPlus, and PNAEq MPNN layers are also included in HydraGNN but omitted to prevent an excessively large figure.

11.1.3 The `'globalAtt'` sub-module

The `'globalAtt'` sub-module implements the GraphGPS architecture. The core `GPSConv` class wraps any MPNN convolutional layer and augments it with global multi-head self-attention, enabling the model to capture long-range interactions beyond the local neighborhood defined by the message-passing layers. The module supports standard multi-head attention and Performer attention for linear-complexity scaling. Laplacian positional encodings and relative positional encodings are integrated to provide structural awareness to the attention mechanism (see Section 12.3 for configuration details).

11.1.4 The `'postprocess'` sub-module

The `'postprocess'` sub-module provides:

- **Output denormalization:** Reverses min-max normalization to map predictions back to the original domain via `output_denormalize()`
- **Feature unscaling:** Reverses per-node scaling for features that were normalized by the number of nodes
- **Visualization:** A comprehensive `Visualizer` class that generates:
  - Parity (scatter) plots of true vs. predicted values for each target
  - Error histograms per variable and per node
  - Conditional mean error plots ($\langle|\epsilon|\rangle$ vs. true value)
  - Training loss history plots for train, validation, and test sets
  - Graph size distribution plots

11.2 THE 'EXAMPLES' MODULE

This module provides a comprehensive set of examples that instantiate HydraGNN models and train them on both open-source and synthetic datasets. The examples cover a wide range of applications:

- **Standard molecular datasets:** `qm9` (Ramakrishnan et al. 2014) (molecular properties), `md17` (molecular dynamics), `qm7x` (Hoja et al. 2021) (extended molecular properties), `zinc` (molecular graphs)
- **Materials datasets:** `alexandria` (Schmidt et al. 2021) (materials database), `lsms` (magnetic alloys), `eam` (embedded atom method for NiNb)
- **Catalysis and materials discovery:** `open_catalyst_2020` (Chanussot et al. 2021), `open_catalyst_2022` (Tran, Shuaibi, Das, et al. 2023), `open_catalyst_2025` (Sahoo et al. 2025), `open_materials_2024` (Barroso-Luque et al. 2024), `open_molecules_2025`, `open_polymers_2026` (Levine, Liesen, Chua, et al. 2025)
- **Interatomic potentials:** `LennardJones` (toy potential), `ani1_x` (Smith et al. 2020) (ANI-1x dataset), `mptrj` (Jain et al. 2013) (Materials Project trajectories)
- **Spectral predictions:** `dftb_uv_spectrum` (UV spectrum prediction)
- **Multi-dataset training:** `multidataset`, `multidataset_deepspeed`
- **Model parallelism:** `multibranch` (multi-branch parallelism for foundation models (Allen et al. 2024; Shiota et al. 2024; Jacobson et al. 2023))
- **Hyperparameter optimization:** `qm9_hpo`, `multidataset_hpo`
- **Non-chemistry applications:** `opf` (optimal power flow), `ising_model` (spin configurations), `homogeneous_graphs`

The user is recommended to create a new folder inside this module when a new HydraGNN model must be trained on a new user-specific dataset.

11.3 THE 'TESTS' MODULE

The Python tests provided in this module aim at testing the different capabilities supported by HydraGNN, and make sure that new changes made to the existing library do not compromise the correct functioning of the other existing capabilities.

As follows, we provide a description of the existing python tests:

- `'test_config.py'`: checks that the syntax of a JSON parsing file is correct
- `'test_graphs.py'`: uses the script `'deterministic_graph_data.py'` to generate a dataset with random graphs (varying number of nodes and randomized node features), serializes the dataset into files in `pickle` format, and trains different MPNN models with and without GraphGPS global attention and edge features. The test fails if at least one of the models does not reach the prescribed accuracy.
- `'test_graphs_graphattr.py'`: tests graph attribute conditioning modes including `concat_node`, FiLM, and `fuse_pool`
- `'test_examples.py'`: runs the examples available in the `examples/` directory to verify their correctness as integration tests

- 'test_atomicdescriptors.py': checks that atomic descriptors can be correctly extracted from the Mendeleev library
- 'test_loss_and_activation_functions.py': checks that all the training loss functions and activation functions supported in HydraGNN can be used to perform the training of a model
- 'test_model_loadpred.py': checks that a pre-trained model can be loaded and used for inference
- 'test_optimizer.py': checks that all the stochastic optimization numerical schemes supported in HydraGNN can be used to perform the training of a model
- 'test_periodic_boundary_conditions.py': checks that periodic boundary conditions are correctly applied to crystal structures, verifying that (1) the graph with PBC has more edges than without PBC, and (2) every additional edge has a length shorter than the radius cut-off
- 'test_rotational_invariance.py': checks that the pre-processing correctly maps two graphs that differ only by rotations and translations into the same pre-processed graph
- 'test_forces_equivariant.py': verifies that equivariant force prediction via compute_grad_energy is functional and that forces transform correctly under rotations
- 'test_forces_equivariant_training.py': integration test for equivariant force training
- 'test_interatomic_potential.py': tests MLIP energy-force loss training pipeline
- 'test_radial_transforms.py': checks that MACE and other models work correctly with Bessel, Gaussian, Agnesi, and Soft distance transforms
- 'test_precision_control.py': validates training under bf16, fp32, and fp64 precision
- 'test_deepspeed.py': tests DeepSpeed integration for distributed training
- 'test_fsdp2_force_grad_regression.py': tests FSDP v2 with force gradient regression
- 'test_datasetclass_inheritance.py': verifies that custom dataset classes can properly inherit from the base dataset classes

11.4 THE 'UTILS' SUB-MODULE

The 'utils' sub-module contains auxiliary functionalities organized into the following categories:

11.4.1 Data loading and dataset classes

HydraGNN provides a hierarchy of dataset classes for loading data from various formats:

- abstractbasedataset.py: Abstract base class defining the common interface for all dataset classes
- abstractrawdataset.py: Abstract class for raw (unprocessed) datasets with graph construction logic
- lsmsdataset.py: Dataset class for LSMS magnetic materials data (FePt, CuAu, FeSi alloys)
- cfgdataset.py: Dataset class for ASE CFG atomic configuration files
- xyzdataset.py: Dataset class for standard XYZ coordinate files
- pickledataset.py: Dataset class for pre-serialized Pickle format data

- `adiosdataset.py`: Dataset class for ADIOS2 high-performance parallel I/O, including an `AdiosWriter` for creating ADIOS2 datasets
- `serializeddataset.py`: Dataset class for pre-serialized datasets
- `distdataset.py`: Dataset class using DDStore for distributed data access across nodes
- `compositional_data_splitting.py`: Stratified data splitting by elemental composition for representative train/validation/test sets

11.4.2 Atomic and edge descriptors

- `descriptors_and_embeddings/atomicdescriptors.py`:
  Provides atomic embeddings derived from the Mendeleev library with support for one-hot encoding. See Section 14.3 for a comprehensive description of the available descriptors and their usage.

11.4.3 Configuration parsing

- `input_config_parsing/config_utils.py`: Parses and validates JSON configuration files via `update_config()`, which auto-fills defaults, computes PNA degree histograms, detects edge dimensions, and validates consistency between architecture and data settings.

11.4.4 Model management

- `model/model.py`: Provides utilities to print model details, save trained models to files, and load pre-trained models. Also contains the loss function selection logic.

11.4.5 Distributed computing

- `distributed/distributed.py`: Provides utilities for DDP setup, process group management, rank/world-size detection across SLURM, LSF, and MPI environments, and backend auto-selection (NCCL, CCL, GLOO). Also includes `SyncBatchNorm` support.

11.4.6 Optimizer and scheduler

- `optimizer/optimizer.py`: Provides all supported optimizers (SGD, Adam, AdamW, Adadelta, Adagrad, Adamax, RMSprop, FusedLAMB) with optional ZeroRedundancy variants. Includes a `ReduceLROnPlateau` scheduler (factor=0.5, patience=5, min_lr=1e-5).

11.4.7 Profiling and tracing

- `profiling_and_tracing/profile.py`: Integration with the PyTorch profiler for performance analysis
- `profiling_and_tracing/time_utils.py`: Timing utilities for measuring computational time in different sections of execution
- `profiling_and_tracing/tracer.py`: Custom tracing module for detailed execution monitoring

12. CONSTRUCTION OF THE HYDRAGNN ARCHITECTURE

12.1 MPNN LAYERS AND FC LAYERS

The HydraGNN architecture is made of the following main components:

- MPNN layers
- Optional FC layers shared across all heads for all predictive tasks
- FC layers specific to each head for a specific predictive task

12.1.1 MPNN LAYERS

The MPNN layers are located at the beginning of the architecture, and are shared across all the predictive tasks. Their purpose is to learn interactions between nodes to create a feature context that can contribute to all the downstream predictive tasks.

HydraGNN supports 13 different message-passing policies that can be selected via the `mpnn_type` key: GIN, PNA, PNAPlus, GAT, MFC, CGCNN, SAGE, SchNet, DimeNet, EGNN, PNAEq, PAINN, and MACE. Each policy implements a different strategy for constructing, aggregating, and updating messages between neighboring nodes; a detailed description is provided in Appendix A.

The number of message-passing layers is controlled by `num_conv_layers`, and the dimensionality of the hidden node features is set by `hidden_dim`. All message-passing layers include batch normalization and a configurable activation function.

12.1.1.1 Invariant Models.

The invariant models (GIN, PNA, PNAPlus, GAT, MFC, CGCNN, SAGE, SchNet, DimeNet) produce scalar node representations that are invariant to rotations and translations of the input graph. These models are suitable for predicting scalar-valued properties such as energies, band gaps, and charges.

12.1.1.2 Equivariant Models.

The equivariant models (EGNN, PNAEq, PAINN, MACE) maintain both invariant scalar features and equivariant vector (or higher-order tensor) features throughout the message-passing process. Equivariant message passing follows the dual-track formulation described in Section 4.2.3, where the invariant and equivariant tracks evolve jointly. These models satisfy physical symmetry constraints even when predicting vectorial quantities, such as interatomic forces.

12.1.2 GRAPH POOLING

For graph-level predictions, a global pooling layer aggregates node-level representations into a fixed-size graph-level embedding. HydraGNN supports three pooling strategies:

- `mean`: Mean pooling (average of all node representations)
- `add` (or `sum`): Sum pooling (sum of all node representations)
- `max`: Max pooling (element-wise maximum across all node representations)

The pooling strategy is configured via the `graph_pooling` key in the `Architecture` section.

12.1.3 FC LAYERS

After the stack of message-passing layers, the architecture bifurcates to create subsequent stacks of FC layers that are specific to each target property. The multi-headed design consists of two types of FC layers:

- **Shared FC layers:** These layers are common to all prediction heads, allowing them to jointly refine the learned representations before task-specific predictions. The number of layers is controlled by `num_sharedlayers`. The dimensionality is controlled by `dim_sharedlayers`, which can be provided either as a single integer (all shared layers have the same width) or as an array of integers whose length equals `num_sharedlayers` (each layer has an individually specified width).
- **Head-specific FC layers:** Each prediction head has its own stack of FC layers dedicated to a single target property. The number of layers is controlled by `num_headlayers`. The dimensionality is controlled by `dim_headlayers`, which can be provided either as a single integer (all head layers have the same width) or as an array of integers whose length equals `num_headlayers` (each layer has an individually specified width).

For node-level predictions, HydraGNN supports three types of output heads via the `type` key:

- `mlp`: A standard multi-layer perceptron applied to each node independently.
- `mlp_per_node`: A separate MLP for each node (useful when nodes have distinct roles).
- `conv`: Additional convolutional layers for the output head.

In Listing 18, we provide an example of the `Architecture` section of the JSON configuration file that defines the construction of a multi-headed HydraGNN architecture. The list of parameters to define inside this section are:

- `mpnn_type`: type of MPNN layers (see Appendix A for all supported types)
- `radius`: radius cut-off used to establish the graph connectivity and pairwise interaction mask for some distance transforms
- `max_neighbours`: upper bound on the degree of a node (maximum number of neighbours)
- `hidden_dim`: number of channels for the MPNN layers
- `num_conv_layers`: number of MPNN layers
- `activation_function`: non-linear activation used throughout the architecture (see Appendix B)
- `graph_pooling`: pooling strategy for graph-level predictions (`mean`, `add`, `max`)
- `output_heads`: type of head, can either be `graph` or `node`
  - `num_sharedlayers`: number of FC layers shared across all heads of the architecture
  - `dim_sharedlayers`: width of the shared FC layers. Accepts either a single integer (uniform width across all `num_sharedlayers` layers) or an array of integers of length `num_sharedlayers` (per-layer widths). Example: `200` or `[200, 100]`.
  - `num_headlayers`: number of FC layers specific to one head of the architecture
  - `dim_headlayers`: width of the head-specific FC layers. Accepts either a single integer (uniform width across all `num_headlayers` layers) or an array of integers of length `num_headlayers` (per-layer widths). Example: `500` or `[1000, 500]`.
  - `type`: for node heads, one of `mlp`, `mlp_per_node`, or `conv`

- `task_weights`: list of scalars that provide the scaling coefficients used to multiply the loss functions of each head in order to define the global loss function minimized during the MTL training

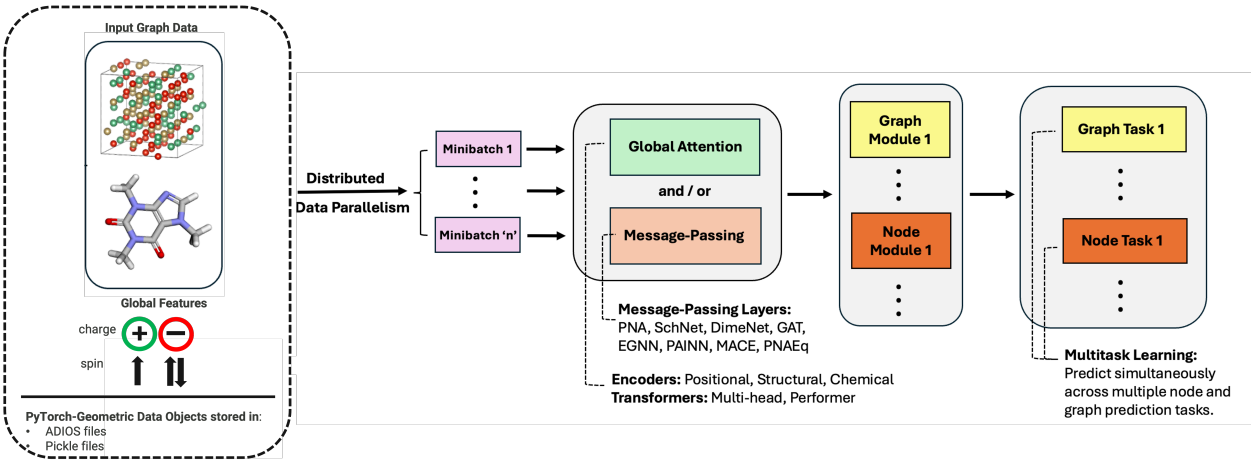

Figure 2. Overview of the HydraGNN workflow and model architecture. Input graph data from ADIOS, pickle, or PyTorch Geometric sources are partitioned into distributed minibatches and processed by a shared backbone composed of message-passing layers and optional global-attention/transformer blocks. The learned representations are then routed to graph-level and node-level modules, followed by task-specific output heads that support simultaneous multi-task prediction across graph and node properties.

Listing 18. Example of the Architecture section of the JSON configuration file

```
"NeuralNetwork": {
        "Architecture": {
            "mpnn_type": "PNA",
            "radius": 7,
            "max_neighbours": 100,
            "hidden_dim": 5,
            "num_conv_layers": 6,
            "activation_function": "relu",
            "graph_pooling": "mean",
            "output_heads": {
                "graph":{
                    "num_sharedlayers": 2,
                    "dim_sharedlayers": 5,
                    "num_headlayers": 2,
                    "dim_headlayers": [50,25]
                },
                "node": {
                    "num_headlayers": 2,
                    "dim_headlayers": [50,25],
                    "type": "mlp"
                }
            },
            "task_weights": [1.0, 1.0, 1.0]
        },
        "Variables_of_interest": {
            ...
        },
        "Training": {
            ...
        }
    },
```

12.2 MACHINE-LEARNED INTERATOMIC POTENTIALS

HydraGNN supports the construction of MLIPs through the `enable_interatomic_potential` flag in the `Architecture` section. When enabled, the model predicts per-atom energies, which are summed to obtain the total energy prediction. Forces are then computed as the negative gradient of the total energy with respect to atomic positions via automatic differentiation:

$$\mathbf{F}_i = -\frac{\partial E_{\text{total}}}{\partial \mathbf{r}_i}, \tag{27}$$

where $E_{\text{total}} = \sum_i E_i$ is the total energy and $\mathbf{r}_i$ is the position of atom i. This approach enforces energy conservation by construction, which is essential for running molecular dynamics simulations.

The combined loss function for MLIP training is:

$$\mathcal{L}_{\text{MLIP}} = w_E \cdot \mathcal{L}_E + w_{E/N} \cdot \mathcal{L}_{E/N} + w_F \cdot \mathcal{L}_F, \tag{28}$$

where $\mathcal{L}_E$, $\mathcal{L}_{E/N}$, and $\mathcal{L}_F$ are the total energy loss, per-atom energy loss, and force loss, respectively, and w_E, $w_{E/N}$, w_F are configurable weight coefficients. The relevant JSON keys are:

- `enable_interatomic_potential`: Boolean to activate MLIP mode

- `energy_weight`: weight for the total energy loss component
- `energy_peratom_weight`: weight for the per-atom energy loss
- `force_weight`: weight for the force loss component

Important: graph pooling for MLIPs

When using MLIP mode with graph-level energy output, the `graph_pooling` must be set to `add` to ensure that the total energy is obtained as the sum of per-atom contributions. Although choosing another pooling mechanism would still allow for the code to run, the predicted outputs would not be physical.

12.3 GLOBAL ATTENTION VIA GRAPHGPS

HydraGNN integrates the GraphGPS (Rampášek et al. 2022) architecture, which augments any local message-passing layer with global multi-head self-attention. This allows the model to capture long-range interactions that may not be reachable within the receptive field of finite message-passing layers.

The GraphGPS layer wraps any MPNN convolutional layer and processes node features through the following sequence:

$$\mathbf{X}^{(l+1)} = \mathrm{Norm}\Big(\mathrm{MLP}(\mathrm{Norm}(\mathrm{GlobalAttn}(\\ \mathrm{Norm}(\mathrm{LocalMPNN}(\mathbf{X}^{(l)}))))\Big). \quad (29)$$

The global attention mechanism supports two modes:

- `multihead`: Standard multi-head self-attention via `torch.nn.MultiheadAttention`
- `performer`: Linear-complexity Performer attention for scalability to large graphs

GraphGPS also incorporates Laplacian positional encoding (LPE) computed from the graph Laplacian to provide structural position information. The relevant configuration keys are:

- `global_attn_engine`: Set to `GPS` to activate (default: `null`)
- `global_attn_type`: Attention mechanism (`multihead` or `performer`)
- `global_attn_heads`: Number of attention heads
- `pe_dim`: Dimensionality of the Laplacian positional encodings

12.4 GRAPH ATTRIBUTE CONDITIONING

HydraGNN supports conditioning the GNN computation on global graph attributes (e.g., temperature, pressure, composition descriptors, charge, or spin) via the `use_graph_attr_conditioning` flag. Three conditioning modes are available:

- `concat_node`: Graph attributes are concatenated to each node's feature vector before message passing.
- `film`: Feature-wise Linear Modulation (FiLM) is applied, where graph attributes parameterize an affine transformation $\gamma \odot \mathbf{x} + \beta$ of the node features.
- `fuse_pool`: Graph attributes are fused into the graph-level representation after global pooling.

The conditioning mode is set via the `graph_attr_conditioning_mode` key in the `Architecture` section.

13. DISTRIBUTED ENVIRONMENT

13.1 DISTRIBUTED DATA PARALLELISM

DDP is a technique used in parallel computing and distributed systems to train machine learning models across multiple devices or nodes (e.g., GPUs, CPUs, or different machines) in a distributed and efficient manner. It is commonly used in DL to accelerate the training process for large models on extensive datasets.

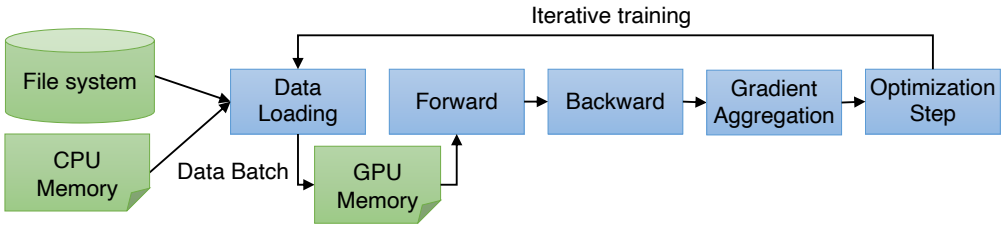

Figure 3. Iterative training with DDP.

DDP is one of the parallel methods commonly used in AI/DL to train models using multiple processors or machines. Each process handles only a subset of the data (or graphs) and executes functions in parallel with others. When information needs to be consolidated, such as for aggregated gradient updates, processes communicate data using *all-gather* operations. Message Passing Interface (MPI) (Gropp et al. 1996; Gabriel et al. 2004) and NVIDIA Collective Communications Library (NCCL) (Li et al. 2020) are well-known communication backends in multi-node, multi-GPU HPC environments.

DDP provides several benefits, such as efficient use of computing resources by providing fault tolerance, and enabling training on large datasets through concurrent processing of batch data. HydraGNN extends PyTorch's DDP implementation (Paszke et al. 2019) with enhancements targeting HPC environments (Lupo Pasini et al. 2025; Md et al. 2021; Sypetkowski et al. 2024; Kong et al. 2026), including automatic backend detection (NCCL for NVIDIA GPUs, CCL for Intel XPUs, GLOO for CPUs) and distributed environment auto-detection across SLURM, LSF, and MPI job schedulers.

Generally, DDP for model training involves the following five steps (Fig. 3):

1. The dataset is divided into smaller, non-overlapping subsets called "batches." A single batch consisting of N samples is loaded into each process (**data loading**).
2. Each process generates predictions for the samples in the batch using its local model (**forward**).
3. Each process assesses the loss between these predictions and the actual values and subsequently computes gradients for the model parameters based on the loss (**backward**).
4. Gradients from all processes are aggregated, producing a consolidated gradient (**gradient aggregation**).
5. Each process updates its local model parameters using the aggregated gradients (**optimization**).

13.1.1 DeepSpeed Integration

HydraGNN integrates with Microsoft DeepSpeed (Microsoft 2020) for memory-efficient distributed training, particularly its ZeRO (Zero Redundancy Optimizer) optimization stages (Rajbhandari et al. 2020). The integration is implemented in `hydragnn/utils/distributed/distributed.py` and is activated via command-line flags (`-deepspeed`) or environment variables.

13.1.1.1 ZeRO Optimization Stages.

DeepSpeed's ZeRO partitions training state across processes to reduce per-device memory. HydraGNN supports all four stages:

- **Stage 0:** No sharding (baseline DDP behavior).
- **Stage 1:** Optimizer state partitioning, where each process stores only $1/N$ of the optimizer states (e.g., Adam's momentum and variance buffers).
- **Stage 2:** Stage 1 + gradient partitioning, where gradients are reduced and partitioned to match the optimizer state partitioning.
- **Stage 3:** Stage 2 + parameter partitioning, where model parameters are also sharded, enabling training of models that exceed the memory of a single device.

The ZeRO stage is configured via the JSON configuration. By default, when the `-zero_opt` flag is used, ZeRO Stage 1 is applied. To use other stages, provide a full DeepSpeed configuration in the `ds_config` section:

Listing 19. DeepSpeed ZeRO configuration in the JSON file

```
"NeuralNetwork": {
    "ds_config": {
        "train_micro_batch_size_per_gpu": 64,
        "gradient_accumulation_steps": 1,
        "zero_optimization": {
            "stage": 2
        }
    }
}
```

13.1.1.2 Training Loop Integration.

When DeepSpeed is enabled, the model is wrapped via `deepspeed.initialize()`, which takes over optimizer management, gradient zeroing, backward pass, and optimizer step. HydraGNN defers these operations to DeepSpeed:

- `zero_grad()` is handled internally by DeepSpeed.
- `model.backward(loss)` replaces the standard `loss.backward()`.
- `model.step()` replaces `optimizer.step()`.

The learning rate scheduler remains managed by HydraGNN (per-epoch scheduling rather than per-step).

13.1.1.3 bf16 with DeepSpeed.

When `bf16` precision is enabled alongside DeepSpeed, HydraGNN disables DeepSpeed's native bf16 engine and instead uses PyTorch's `torch.autocast` mechanism. This ensures consistent precision behavior between DeepSpeed and non-DeepSpeed code paths.

13.1.1.4 Checkpointing.

DeepSpeed uses its native checkpoint mechanism (`model.save_checkpoint()` and `model.load_checkpoint()`) instead of PyTorch's `torch.save`, which correctly handles sharded optimizer states across stages.

13.1.1.5 Launching.

DeepSpeed is initialized via `deepspeed.init_distributed()` rather than the standard `dist.init_process_group()`. On HPC systems, jobs are typically launched with `srun` (SLURM) rather than the DeepSpeed launcher:

Listing 20. Launching DeepSpeed training on an HPC cluster

```
srun -N${SLURM_NNODES} -n$((SLURM_NNODES*4)) \
    --ntasks-per-node=4 --gpus-per-task=1 \
    python train.py --deepspeed --inputfile=config.json
```

13.1.2 Fully Sharded Data Parallelism (FSDP)

HydraGNN supports both FSDP v1 (wrapper-based) and FSDP v2 (composable API) from PyTorch (Zhao et al. 2023), controlled via environment variables. FSDP shards model parameters, gradients, and optimizer states across processes, enabling training of models that exceed the memory of a single device.

13.1.2.1 Configuration.

FSDP is controlled by three environment variables:

Table 5. Environment variables for FSDP configuration.

Variable	Default	Description
`HYDRAGNN_USE_FSDP`	`0`	Set to 1 to enable FSDP
`HYDRAGNN_FSDP_VERSION`	1	FSDP version (1 or 2)
`HYDRAGNN_FSDP_STRATEGY`	`FULL_SHARD`	Sharding strategy

13.1.2.2 Sharding Strategies.

FSDP v1 supports all PyTorch sharding strategies: `FULL_SHARD` (full sharding of parameters, gradients, and optimizer states), `SHARD_GRAD_OP` (gradient and optimizer sharding only), `NO_SHARD`, `HYBRID_SHARD` (intra-node sharding), and `HYBRID_SHARD_ZERO2`. FSDP v2 supports `FULL_SHARD` and `SHARD_GRAD_OP`.

13.1.2.3 FSDP v1 vs. v2.

FSDP v1 wraps the entire model using `FSDP(model, sharding_strategy=...)`. FSDP v2 uses the composable API via `fully_shard(model, reshard_after_forward=...)` and wraps the result in a compatibility layer (`ModuleCompat`) that provides the `model.module` attribute expected by HydraGNN's training loop.

13.1.2.4 Checkpoint Saving.

With FSDP v1, the full model state is gathered on rank 0 using `FSDP.state_dict_type()` with `FULL_STATE_DICT` before saving. With FSDP v2, `model.state_dict()` is called directly, as the composable API handles state collection transparently.

13.1.2.5 FSDP2 and Force Computation.

When using FSDP v2 with MLIP mode (`compute_grad_energy: true`), a workaround is applied automatically: parameter resharding is temporarily disabled before the forward pass (so all parameters remain materialized during `torch.autograd.grad` for force computation) and re-enabled before the backward pass. This ensures correct gradient computation for *single-head* models without user intervention. However, significant unresolved incompatibilities remain when combining FSDP with multi-headed MLIP models; see Section 13.1.4 for a detailed discussion.

13.1.2.6 Usage.

Listing 21. Enabling FSDP v2 with full sharding

```
export HYDRAGNN_USE_FSDP=1
export HYDRAGNN_FSDP_VERSION=2
export HYDRAGNN_FSDP_STRATEGY=FULL_SHARD
torchrun --nproc_per_node=4 train.py --inputfile=config.json
```

13.1.3 DeepSpeed vs. FSDP: Choosing a Sharding Backend

Both DeepSpeed ZeRO and PyTorch FSDP reduce per-device memory by sharding optimizer states, gradients, and (optionally) parameters across ranks. Because they address the same problem, users must choose one or the other; they cannot be combined. Table 6 summarizes the practical trade-offs within the HydraGNN context.

Table 6. DeepSpeed ZeRO vs. PyTorch FSDP in HydraGNN.

Criterion	DeepSpeed ZeRO	PyTorch FSDP
Dependency	External package (`deepspeed`); requires matching CUDA toolkit and compiler	Built into PyTorch ($\geq$ 1.12 for v1, $\geq$ 2.4 for v2); no extra install
Sharding granularity	Four discrete stages (0–3) selected via a single integer	Strategy enum (`FULL_SHARD`, `SHARD_GRAD_OP`, `HYBRID_SHARD`, etc.)
Offloading	CPU and NVMe offloading for optimizer states and parameters (ZeRO-Infinity)	CPU offloading supported in v1; limited in v2
Mixed precision	Native fp16/bf16 engine with loss scaling; HydraGNN uses `torch.autocast` instead for consistency	Integrates with PyTorch's native `autocast` and `MixedPrecision` policy
Activation checkpointing	Built-in `deepspeed.checkpointing` API	Uses PyTorch's `checkpoint_wrapper` or manual `torch.utils.checkpoint`
Task parallelism	Not directly composable with `MultiTaskModelMP`	Composable with task-parallel model parallelism (Section 13.1.5); supports per-group FSDP wrapping and `DualOptimizer`
Checkpoint format	Proprietary sharded format; requires `model.load_checkpoint()`	Standard `state_dict`; portable across DDP, FSDP, and single-GPU
MLIP force computation	Compatible; no special handling needed	FSDP v2 requires automatic reshard workaround (see Section 13.1.2)
Launcher	Typically `srun` on HPC; DeepSpeed launcher also available	Standard `torchrun` or `srun`

13.1.3.1 Recommendations.

- Use **FSDP** when task parallelism (`MultiTaskModelMP`) is needed, when a minimal dependency footprint is preferred, or when checkpoint portability matters.
- Use **DeepSpeed** when ZeRO-Infinity offloading to CPU/NVMe is required, or when an existing DeepSpeed workflow is already established.
- For most HydraGNN workloads that fit in GPU memory with DDP alone, neither is necessary. Enable sharding only when per-device memory becomes a bottleneck.

13.1.4 Known Limitations: FSDP with MLIP Force Computation

Caution: Unresolved Incompatibilities

Combining FSDP (v1 or v2) with multi-headed MLIP models that compute interatomic forces is **not supported** in HydraGNN. The automatic differentiation mechanism used to derive forces from energy predictions is fundamentally at odds with FSDP's parameter sharding. Users should fall back to standard DDP for multi-headed MLIP workloads.

HydraGNN computes interatomic forces as the negative gradient of predicted energy with respect to atomic positions:

$$\mathbf{F}_i = -\frac{\partial E}{\partial \mathbf{r}_i}, \tag{30}$$

where E is the total energy predicted by the model and $\mathbf{r}_i$ is the position vector of atom i. This is implemented via `torch.autograd.grad()`, which requires that *all* model parameters involved in the forward pass remain materialized on the local device so their computational graph entries are accessible for gradient computation. This requirement conflicts with FSDP's core mechanism of sharding (and lazily materializing) parameters across ranks.

13.1.4.1 The Single-Head Workaround (FSDP v2 only).

For *single-head* MLIP models (`num_heads = 1`), HydraGNN implements an automatic workaround in the training loop. Before the forward pass, parameter resharding is temporarily disabled via `set_reshard_after_backward(model, False)`, which forces FSDP v2 to keep all parameters materialized on every rank. After force computation completes, resharding is re-enabled before the backward pass:

1. **Before forward:** Disable resharding → all parameters materialized.
2. **Forward pass + force computation:** `torch.autograd.grad()` succeeds.
3. **Before backward:** Re-enable resharding → FSDP can aggregate gradients normally.

This workaround is validated by the regression test `tests/test_fsdp2_force_grad_regression.py` for the EGNN, DimeNet, SchNet, MACE, PaiNN, and PNA-Eq architectures with the `SHARD_GRAD_OP` strategy.

13.1.4.2 Why Multi-Headed MLIP Models Fail.

The `energy_force_loss()` method enforces a hard constraint:

Listing 22. Single-head assertion in force computation

```
assert self.num_heads == 1, \
    "Force␣predictions␣require␣exactly␣one␣head."
```

This restriction exists because `torch.autograd.grad()` computes the gradient of a *scalar* output with respect to input positions. With multiple heads predicting different quantities (e.g., energy on one head, charge on another), the computational graph branches through all heads simultaneously. The resulting interactions create several problems when combined with FSDP:

1. **Ambiguous gradient source:** With multiple heads, the autograd graph contains contributions from all heads. Extracting the force as $-\nabla_{\mathbf{r}} E$ from a single head requires isolating that head's subgraph, which is not straightforward when FSDP has restructured the parameter layout.

2. **Parameter ownership conflicts:** FSDP shards parameters at the module level. In a multi-headed architecture, decoder parameters for different heads may be sharded across different ranks. During `torch.autograd.grad()`, the gradient computation walks the full computational graph, but sharded parameters on other ranks are not accessible, resulting in missing gradient entries or `RuntimeError`.
3. **Resharding timing:** The single-head workaround (disable resharding → forward → autograd.grad → re-enable → backward) relies on a clean separation between force computation and the backward pass. With multiple heads contributing to the loss, this separation breaks down, and the backward pass for non-force heads may need parameters that have already been resharded.

13.1.4.3 FSDP v1: Initial Attempt and Failure.

FSDP v1 was the first sharding backend tested with multi-headed MLIP models. It lacks a composable `set_reshard_after_backward()` API: the v1 wrapper manages parameter materialization internally through its forward/backward hooks, and there is no supported mechanism to defer resharding during intermediate `autograd.grad()` calls. As a result, `torch.autograd.grad()` fails to traverse the computational graph correctly because parameters have already been freed (resharded) by FSDP's post-forward hooks before force computation begins. Even single-head MLIP models experience failures with FSDP v1 when computing forces. This combination is **unsupported**.

13.1.4.4 FSDP v2: Attempted Solution and Partial Success.

FSDP v2's composable API (`fully_shard()`) was explored as a potential solution, since it exposes a `set_reshard_after_backward()` method that allows temporary control over parameter materialization. For *single-head* MLIP models, this workaround succeeds: disabling resharding before the forward pass keeps all parameters materialized for the subsequent `autograd.grad()` call (see the single-head workaround above).

However, when this same approach was applied to *multi-headed* MLIP models, FSDP v2 also failed. The fundamental issue is that multiple decoder heads create branching computational graphs that interact with FSDP's parameter management in ways the workaround cannot resolve:

- Even with resharding disabled, the autograd graph for multi-headed models contains entangled contributions from all heads. Isolating the energy head's gradient path from the other heads' paths is not feasible through FSDP configuration alone.
- The `autograd.grad()` call attempts to differentiate through parameters belonging to all heads, but the gradient computation produces incorrect or incomplete results when FSDP's internal bookkeeping (reference counting, gradient accumulation buffers) was designed for the standard `loss.backward()` path rather than selective `autograd.grad()`.
- Attempts to restructure the forward pass to compute force-related gradients before other heads' contributions did not resolve the issue, as the shared encoder's computational graph is common to all heads and cannot be cleanly partitioned.

These incompatibilities remain **unresolved**. Multi-headed MLIP models requiring force computation must use standard DDP.

13.1.4.5 Practical Guidance.

- **Single-head MLIP with FSDP v2:** Supported. Use `HYDRAGNN_FSDP_VERSION=2` with `FULL_SHARD` or `SHARD_GRAD_OP`. The workaround is applied automatically. Validated for EGNN, DimeNet, SchNet, MACE, PaiNN, and PNA-Eq.

- **Multi-headed MLIP with FSDP (v1 or v2): Not supported.** Use standard DDP instead.
- **Single-head MLIP with FSDP v1: Not supported.** No reshard workaround is available. Use FSDP v2 or fall back to DDP.
- **Multi-headed non-MLIP models with FSDP:** Fully supported. When `compute_grad_energy` is `false`, FSDP sharding does not interfere with the standard loss computation.
- **DeepSpeed with MLIP:** Compatible without restrictions, as DeepSpeed ZeRO does not interfere with `torch.autograd.grad()` in the same way.

Table 7 summarizes the compatibility matrix.

Table 7. Compatibility of sharding backends with MLIP force computation.

Configuration	DDP	FSDP v1	FSDP v2
Single-head MLIP (forces)	✓	×	✓†
Multi-headed MLIP (forces)	✓	×	×
Multi-headed non-MLIP	✓	✓	✓
Single-head non-MLIP	✓	✓	✓

†Automatic reshard workaround applied; validated for `SHARD_GRAD_OP` and `FULL_SHARD`.

13.1.5 Model Parallelism

For multi-branch architectures with many prediction heads or heterogeneous datasets, HydraGNN supports a *task-parallel* model parallelism strategy via the `MultiTaskModelMP` class in `hydragnn/models/MultiTaskModelMP.py`. This strategy is fundamentally different from the traditional model parallelism approaches used in large language models and other DL frameworks.

13.1.5.1 Distinction from Traditional Approaches.

Table 8 compares HydraGNN's task-parallel strategy with pipeline parallelism and tensor parallelism.

Table 8. Comparison of model parallelism strategies.

Aspect	Task Parallel (HydraGNN)	Pipeline Parallel	Tensor Parallel
What is split	Decoder heads across branch groups; encoder replicated	Model layers across sequential stages	Individual layer weights across ranks
Communication	All-reduce encoder grads (all ranks); decoder grads within branch group	Micro-batch activations forwarded between stages	All-reduce within each split layer
Data flow	Each branch group processes its own dataset	Same data flows through all stages	Same data on all ranks
Idle time	None; all ranks active simultaneously	Pipeline bubble at start and end of each batch	None
Use case	Multi-task / multi-dataset GNN with many heads	Very deep models that exceed single-device memory	Very wide layers that exceed single-device memory

In traditional **pipeline parallelism**, the model is partitioned layer by layer into sequential stages, each placed on a different device. Activations are forwarded from one stage to the next, creating *pipeline bubbles* where some devices are idle while waiting for upstream activations or downstream gradients (Shoeybi

et al. 2019). In **tensor parallelism**, individual weight matrices within a single layer are split across devices, requiring intra-layer all-reduce operations at every forward and backward step.

HydraGNN's **task-parallel** strategy takes a different approach driven by the multi-task GNN architecture. The encoder (shared GNN message-passing backbone) is *replicated* across all ranks using standard DDP or FSDP, ensuring that every rank computes the same encoded node representations. The decoder (prediction heads) is *partitioned* so that each subset of ranks handles only one branch (i.e., one dataset or task). Non-local branches are deleted from each rank's decoder, reducing memory usage proportionally to the number of branches. All ranks are active at all times, there are no pipeline bubbles, and different branch groups can process different datasets simultaneously.

13.1.5.2 Architecture.

`MultiTaskModelMP` wraps a base HydraGNN model and decomposes it into two parts:

- `EncoderModel`: contains the embedding layer and all graph convolution (message-passing) layers. Wrapped with DDP or FSDP using the `WORLD` process group, so encoder gradients are synchronized across all ranks.
- `DecoderModel`: contains the graph pooling, shared layers, and head-specific MLPs. Wrapped with DDP or FSDP using a *branch-specific subgroup*, so decoder gradients are synchronized only among ranks handling the same branch.

13.1.5.3 Process Group Creation.

Ranks are divided into branch groups using one of two methods:

- **DeviceMesh** (`-use_devicemesh`): creates a 2D mesh where one dimension indexes branches and the other indexes within-branch ranks. This requires uniform partitioning (equal number of ranks per branch).
- **Manual subgroups**: allows non-uniform partitioning by specifying the number of ranks per branch explicitly. This is useful when branches have different computational costs.

13.1.5.4 DualOptimizer.

When using FSDP, each FSDP-sharded module (encoder, decoder) requires its own optimizer to correctly handle sharded optimizer states. The `DualOptimizer` class wraps two optimizers (one for the encoder, one for the decoder) into a single optimizer interface compatible with HydraGNN's training loop:

Listing 23. Using DualOptimizer with task-parallel FSDP

```
optimizer1 = AdamW(model.encoder.parameters(), lr=1e-3)
optimizer2 = AdamW(model.decoder.parameters(), lr=1e-3)
optimizer = DualOptimizer(optimizer1, optimizer2)
```

13.1.5.5 Usage.

Task parallelism is enabled via the `-task_parallel` flag:

Listing 24. Running task-parallel multi-branch training

```
# 4 branches on 8 GPUs (2 GPUs per branch, uniform)
torchrun --nproc_per_node=8 train.py \
    --task_parallel --use_devicemesh \
    --modellist branch0 branch1 branch2 branch3
```

```
# Non-uniform: 4 GPUs for branch0, 2 each for others
mpirun -np 8 python train.py \
    --task_parallel --process_list 4 2 2
```

13.2 SCALABLE INPUT/OUTPUT DATA MANAGEMENT

Various methods have been developed to reduce overhead in executing distributed training steps and ensure seamless integration for optimal performance, such as data loading optimizations with latency hiding on HPC systems.

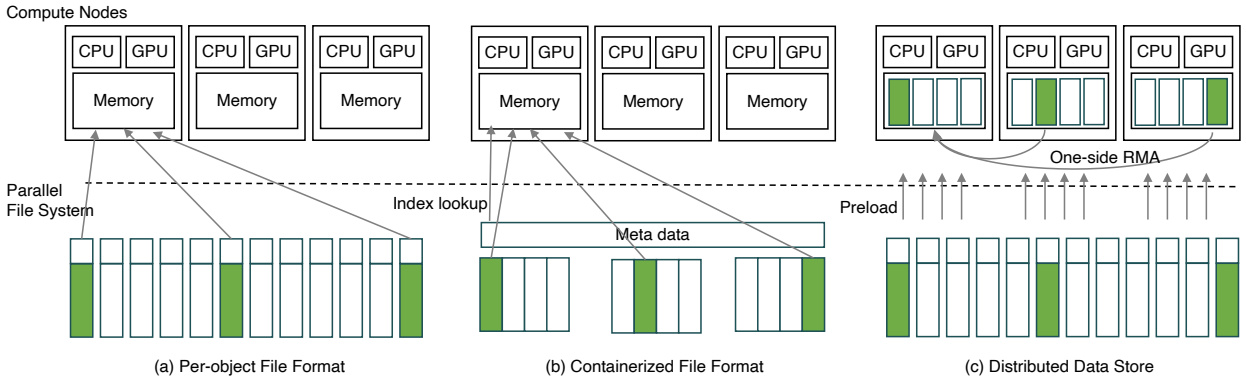

Figure 4. Data storing strategy and its common I/O access patterns in DL: (a) Per-object file format, (b) Containerized file format, (c) Distributed data store.

13.2.1 Per-object File Format

The per-object file format serializes the graph object structures and stores one sample per file. This is one of the simpler approaches for storing graph data, but exerts significant overhead on the underlying parallel file system as the number of samples becomes large. Additionally, as training is performed using tens of thousands of processes, concurrent access to the large number of files from all processes causes a severe I/O bottleneck.

13.2.2 Containerized File Format

Fig. 4(b) shows an alternative method of storing data using containerized file format libraries such as HDF5, ADIOS (Godoy et al. 2020), WebDataset, and TFRecord. In this approach, multiple samples are stored in a single file which reduces the overhead on the file system. The data management library manages metadata on behalf of the user and provides the API to store and retrieve particular data samples. However, DL training requires reading samples in a random or shuffled order to enable unbiased training. Frequent, random, non-sequential I/O accesses to containerized data can lead to a large number of accesses to the file system, which is very inefficient.

13.2.3 In-memory Data Caching

Irrespective of the file format used for storing training data, some solutions involve reading the entire dataset into device memory or node-local storage systems such as NVMe devices. However, these options may not be feasible for large datasets. The size of these datasets can surpass the capacity of a single node's memory or its node-local storage. Moreover, several HPC resources (including some of the existing US-DoE supercomputers) are not endowed with NVMe devices.

13.2.4 Distributed Data Store (DDStore)

To address challenges associated with efficient reading of data for large-scale DL/GNN, HydraGNN includes DDStore (Choi et al. 2023), a distributed data store that specifically addresses random, read-oriented, global shuffle operations.

13.2.4.1 Design.

The design of DDStore is driven by two main performance considerations:

1. Can we minimize access to the file system during the shuffling steps and make in-memory data accessible to other nodes?
2. How do we design fast, efficient, and portable communication mechanisms to provide high-performance shuffling operations for DL?

DDStore splits data into chunks and stores them in the device memory of compute nodes similarly to data sharding. The dataset is read from the file system and distributed across the device memories of compute nodes. All subsequent accesses to samples in the dataset are made via in-memory read transactions. To reduce communication overhead, DDStore uses data replication to maintain multiple copies of the dataset in memory. DDStore internally partitions application processes into groups that are each assigned a replica. This hierarchical design prevents communication bottlenecks. Finally, DDStore uses low-latency MPI Remote Memory Access (RMA) (Dinan et al. 2016), known as one-sided communication, to provide fast, non-blocking read operations that are portable across different systems and architectures (Fig. 4c).

We formally define DDStore as:

$$DS = (c, w, f)$$

where c is the number of chunks that a dataset is striped into, w represents the store *width* that controls the degree of replication of data, and f represents the communication framework used for transferring data between processes.

13.2.4.2 Chunking.

DDStore uses chunking to split a dataset and distribute the chunks evenly amongst nodes. The number of chunks c depends on the total size of the dataset T and the width w:

$$c = T/w$$

By default, for a training using N processes, DDStore stripes the data into N chunks and places one chunk on each process.

13.2.4.3 Replication.

The width w controls the degree of replication of data chunks. DDStore divides application processes internally into sub-groups where each sub-group holds a full replica of the dataset. The number of replicas r is:

$$r = N/w$$

For example, with $N = 1024$ and $w = 128$, DDStore creates 8 groups of 128 processes each. Every process group holds a full replica of the dataset. Processes within a group communicate only with each other to exchange data. By default, $w = N$, which creates a single replica of the dataset striped evenly over all processes. Smaller values for width can help reduce communication bottlenecks by distributing requests more evenly, at the cost of increased memory consumption.

13.2.4.4 Communication.

MPI's RMA one-sided library functions are used for DDStore's communication layer. In contrast to the tightly coupled MPI send-receive paradigm, MPI's RMA minimizes the target process's involvement, helping to increase throughput.

13.2.4.5 Architecture.

DDStore is composed of four components:

1. A **data preloader**, which reads data in various formats from a parallel file system and loads it into memory.
2. A **data registry** that manages the index of data chunks across processes.
3. A **data loader** that reads the next batch of data from other processes.
4. A **communication layer** that leverages one-sided RMA to asynchronously fetch data from remote processes.

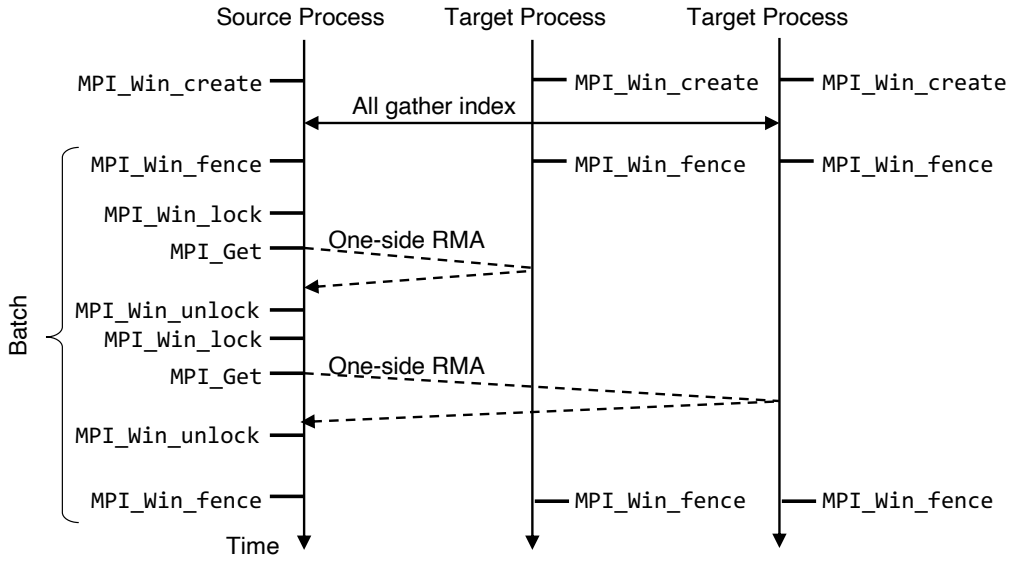

Figure 5. Example walk-through of DDStore's RMA operations for a batch size of 2.

During the registration step, each process registers its memory region containing the data by calling MPI's `MPI_Win_create` function. To fetch data, processes issue one-sided `MPI_Get` operations with read lock (`MPI_Win_lock` with `MPI_LOCK_SHARED`) and fence synchronization (`MPI_Win_fence`) as lightweight contention-avoiding methods. The sequence of operations involved in MPI RMA is shown in Figure 5.

DDStore is integrated into PyTorch's data library by creating subclasses that inherit from `torch.utils.data.Dataset`. This enables PyTorch's data loader `torch.utils.data.DataLoader` to directly interact with the distributed dataset. The dataset classes extend basic data readers for commonly used data formats and integrate them with DDStore for preloading, as well as MPI RMA registration and fetching.

13.3 PROFILING

13.3.1 Timer Class

The Python script `hydragnn/utils/profiling_and_tracing/time_utils.py` provides an implementation of a `Timer` class that measures the time spent in different sections of the code. Some timers to measure the performance of HydraGNN during the training are already implemented. The `Timer` allows the user to implement new timers with a customized name to measure additional sections of the code. The timers instantiated with the `Timer` class collect the minimum, the maximum, and the average time across all processes when the code is executed in a distributed computing manner. The amount of printouts used to describe the performance of HydraGNN across multiple processes can be tuned using the verbosity level described in Section 5.4.

13.3.2 Profiler Class

The Python script `hydragnn/utils/profiling_and_tracing/profile.py` implements a `Profiler` class that inherits from `torch.profiler.profile`. Its use can be activated by including the `Profile` sub-section in the `NeuralNetwork` section of the JSON file as shown below:

Listing 25. Enabling the profiler via the JSON configuration file

```
"NeuralNetwork": {
    "Profile": {"enable": 1},
    "Architecture": {
        ...
    },
    "Variables_of_interest": {
        ...
    },
    "Training": {
        ...
    }
},
```

For more details about PyTorch profilers, see https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html.

13.3.3 Omnistat Telemetry on AMD GPUs

The SC26 multi-dataset examples in HydraGNN also demonstrate node-level telemetry collection with AMD's Omnistat package. In particular, the Frontier job scripts in `examples/multidataset_hpo_sc26` wrap the training run with the Omnistat user-mode monitor rather than modifying HydraGNN itself. This provides a practical way to collect hardware telemetry during large-scale training and hyperparameter optimization campaigns.

The workflow used in the example is simple:

1. Load the `omnistat-wrapper` module.
2. Point `OMNISTAT_CONFIG` to the repository configuration file `omnistat.hydragnn-external-fp64.config`.
3. Start Omnistat before launching the HydraGNN training script.
4. Stop Omnistat after the training or HPO run finishes.

Listing 26. Wrapping a Frontier training run with Omnistat telemetry

```
ml use /sw/frontier/amdsw/modulefiles/
ml omnistat-wrapper
export OMNISTAT_CONFIG=$HYDRAGNN_ROOT/omnistat.hydragnn-external-fp64.config

${OMNISTAT_WRAPPER} usermode --start --interval 15

srun ... python -u examples/multidataset_hpo_sc26/gfm_mlip_all_mpnn.py ...

${OMNISTAT_WRAPPER} usermode --stop
```

In the checked-in Omnistat configuration, HydraGNN enables both the ROCm SMI collector and the rocprofiler collector. In practice, this means that a Frontier run can be instrumented for GPU power and memory-related telemetry through ROCm SMI, while rocprofiler is used to capture compute-activity counters such as active GPU cycles, floating-point instruction counts, and HBM traffic. The configuration also enables RMS/network collection and routes data to an external VictoriaMetrics endpoint.

This integration should be viewed as *external telemetry* rather than an internal HydraGNN profiler. HydraGNN does not currently consume Omnistat metrics inside the training loop or use them for scheduling decisions; instead, Omnistat is launched by the batch script so that system telemetry can be analyzed alongside the training logs after the job completes. This is especially useful when comparing model architectures, precision modes, FSDP vs. DDP configurations, or DeepHyper trials on AMD GPU systems.

For large-scale hyperparameter optimization, this same approach can be used to associate telemetry with the individual model evaluations launched by DeepHyper. The SC26 example includes a dedicated `job-omnistat-deephyper.sh` script, showing that Omnistat can be enabled around a DeepHyper campaign so that GPU power, memory/compute activity, and related system counters are recorded while many HPO trials are executed concurrently across the machine. This can be valuable when comparing candidate configurations not only by validation error or training time, but also by their hardware efficiency and resource footprint.

At present, the Omnistat usage in the repository is specific to the Frontier-oriented SC26 job scripts. Users on other systems should treat these scripts as templates and adapt the module loads, Omnistat installation path, and output/query backend to match their site configuration.

13.3.4 Fine-Grained Energy Profiling

While Omnistat provides high-level telemetry for the entire job, the `Tracer` class found in `hydragnn/utils/profiling_and_tracing/tracer.py` instruments HydraGNN for fine-grained energy and time profiling of user-selected code blocks, yielding detailed insight into the energy and time bottlenecks of individual stages of execution. When distributed multi-GPU training is used, a separate log file is produced for each rank, recording the cumulative energy consumption of every instrumented block.

To enable code-level energy monitoring, set the environment variable `HYDRAGNN_TRACE_LEVEL=1` and instrument the code as shown in the example in Listing 27.

Listing 27. Instrumenting HydraGNN for energy monitoring.

```
    import hydragnn.utils.profiling_and_tracing.tracer as tr
    # Replace ROCMTracer with NVMLTracer for NVIDIA GPUs
    # or XPUTracer for Intel GPUs
    tr.initialize(["GPTLTracer", "ROCMTracer"])
    tr.enable()
```

```
    tr.start("train")
    train_loss, train_taskserr = train(...)
    tr.stop("train")
    tr.disable()
    tr.save(log_file_path)
```

The main training loop in `hydragnn/train/train_validate_test.py` is already instrumented to record the energy consumed by the data-loading, forward-pass, and backward-pass functions but the user has the freedom to add additional markers if required.

14. DATASET CLASSES

The dataset classes are separated in two categories:

- Dataset classes that read data from raw input files and convert it into standardized formats accepted by HydraGNN
- Dataset classes that load pre-standardized data and make it ready for HydraGNN training

The HydraGNN routines for both categories of dataset classes are structured according to an object-oriented programming framework. An illustration of the object-oriented programming framework for data reading and loading routines is provided in Figure 6. Dataset classes for reading of data from raw files are colored in orange, and dataset classes for loading pre-standardized data are colored in green.

The class inheritance for both categories of dataset classes stems from the same common parenting abstract class called `AbstractBaseDataset`. This class provides the backbone structure of methods for all the derived classes. The user can customize the level of inheritance layers before fully specifying the complete set of methods for the data class.

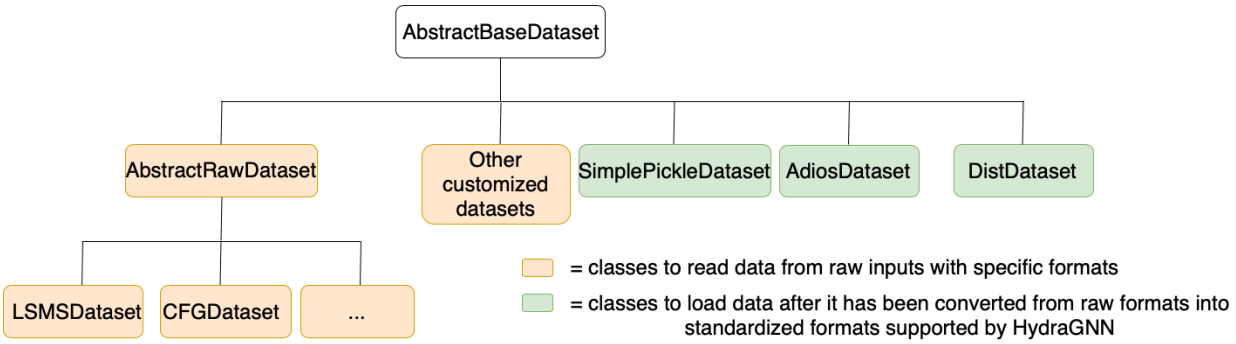

Figure 6. Object-oriented organization of data classes to load data from raw input files and after they have already been standardized into readable formats for HydraGNN.

HydraGNN provides two different methodologies to convert raw files into formats that can be readily passed to the HydraGNN model for training. The first methodology automates the read of data from specific raw data formats and creates `pickle` files with `.pkl` format. This methodology uses the class `SimplePickleWriter` to create `pickle` files and uses the class `SimplePickleDataset` to load the pre-standardized data. The second methodology creates `ADIOS` binary files. This methodology uses the class `AdiosWriter` to create an `ADIOS` binary file and uses the class `AdiosDataset` to load the pre-standardized data. This is also illustrated in Figure 7.

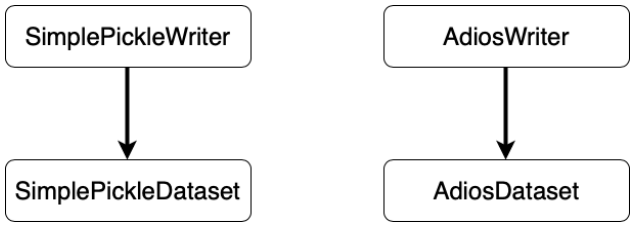

Figure 7. Sequence of data classes to use to convert data in pickle or ADIOS format, and load it in the respective pre-standardized formats.

14.1 OBJECT-ORIENTED PROGRAMMING ORGANIZATION OF DATASET CLASSES

HydraGNN provides capabilities to load customized data from files with different formats. Once loaded, the file is converted into `pickle` or `ADIOS` formats that are easier to manage on HPC resources. Once the datasets are converted, data loader capabilities are used to feed the data into the HydraGNN model for training.

14.1.1 AbstractBaseDataset

The `AbstractBaseDataset` class defines the signature of fundamental abstract methods that all subclasses must implement to provide data import functionalities for specific data formats and to adopt specific I/O data management techniques. The fundamental abstract methods are:

- `get`: fetch data samples from the dataset and feed it to HydraGNN for training
- `len`: retrieve the number of samples contained in the dataset
- `apply`: execute a generic transformation on every data sample contained in the dataset
- `__len__`, `__getitem__`, `__iter__`: Python special methods for standard dataset access

14.1.2 AbstractRawDataset

The `AbstractRawDataset` class implements some methods that are common to all the datasets independently of their specific format:

- `__load_raw_data`: iterates over the raw files that describe every sample of a dataset, and for each data sample calls `transform_input_to_data_object_base` to import the data samples into a `torch_geometric.data.Data` object
- `__normalize_dataset`: normalizes the output features within the range [0, 1] to help stabilize the training
- `__scale_features_by_num_nodes`: rescales the value of the output features based on the number of nodes in the graph
- `__build_edge`: establishes the connectivity among nodes of the graph and builds edge features

The implementation of the method `transform_input_to_data_object_base` depends on the specific data format, and is thus delegated to any child class that inherits from `AbstractRawDataset`.

14.1.3 Concrete Dataset Classes

- **AdiosDataset**: inherits from `AbstractBaseDataset`. Reads information written in an ADIOS file and provides graph objects of the class `torch_geometric.data.Data`. This class supports caching, shared memory, and DDStore.
- **CFGDataset**: inherits from `AbstractRawDataset`. Imports data saved in files with `.cfg` format (ASE atomic configuration files) and converts it into `torch_geometric.data.Data` objects.
- **XYZDataset**: inherits from `AbstractRawDataset`. Imports data from standard XYZ coordinate files.
- **LSMSDataset**: inherits from `AbstractRawDataset`. Imports data from files generated as output by the quantum mechanics code LSMS-3 and converts it into `torch_geometric.data.Data` objects.
- **DistDataset**: inherits from `AbstractBaseDataset`. Distributes datasets according to the DDStore technique (see Section 13.2.4).
- **SimplePickleDataset**: loads pre-standardized data from pickle files.
- **SerializedDataset**: loads pre-serialized datasets.

14.2 LSMS DATA FORMAT

14.2.1 LSMS output – `info_evec_out`

LSMS generates a file called `info_evec_out`. The first line of this file has three columns and lists:

1. total energy
2. band energy
3. Fermi energy

Per Atom Lines

The rest of `info_evec_out`, beginning with the second line, contains one line for each site in the system. Each line has 18 columns with the following information:

- Atomic number
- Site index (starting with 0 for the first site)
- x, y, z components of the site coordinate
- Electron charge associated with this site (counted with a positive sign)
- Magnetic moment associated with this site in μ_B
- x, y, z components of the magnetic moment direction
- Site dependent magnetic moment direction mixing parameter (currently always -1)
- x, y, z components of the magnetic constraining field
- Shift of spin-up and spin-down potential
- x, y, z components of the new magnetic moment direction

14.2.2 Local Atom Data File – `localAtomData`

If the input files for LSMS specify a filename for `localAtomData`, this file is written in a format similar to `info_evec_out`, but with somewhat different contents.

The first line has four columns:

1. total energy
2. band energy
3. Fermi energy
4. electrostatic energy

Per atom lines contain 16 columns with the following information:

- Atomic number
- Site index (starting with 0 for the first site)
- x, y, z components of the site coordinate
- Electron charge associated with this site (counted with a positive sign)
- Magnetic moment associated with this site in μ_B

- x, y, z components of the magnetic moment direction
- x, y, z components of the magnetic constraining field
- Shift of spin-up and spin-down potential
- Local site volume (in units of a_0^3)
- Local site energy

Currently, the `LSMSDataset` class follows the format explained in Section 14.2.1. However, it can be easily customized to follow the format explained in Section 14.2.2.

14.3 ATOMIC AND EDGE DESCRIPTORS

The atomic and edge descriptors help discriminate different types of nodes, which can improve the expressivity of the HydraGNN model. The atomic descriptors are included as columns in the two-dimensional tensor stored in `data.x` whose number of rows is equal to the number of nodes in the graph. The edge descriptors are added as rows to the tensor `data.edge_attr`, which contains the descriptors to discriminate every edge that connects two nodes of the graph.

The atomic descriptors available in HydraGNN are provided through the `mendeleev` library (*mendeleev*). The list of atomic descriptors includes:

- atomic number
- group
- period
- covalent radius
- electron affinity
- atomic volume
- atomic weight
- electronegativity
- valence electrons
- ionization energies

Each property can also be converted into a one-hot representation. For integer-valued quantities, the length of the one-hot encoding vector is dictated by the maximum attainable value. For real-valued quantities, the conversion into one-hot encoding is performed by splitting the range of values into discrete bins.

To extract tabulated information about each atom element and use it as descriptors, first an object of the `atomicdescriptors` class must be instantiated. An example is provided in Listing 28.

Listing 28. Instantiating atomic descriptors with standard and one-hot encoding

```
atomicdescriptor = atomicdescriptors(
    "./embedding.json",
    overwritten=True,
    element_types=["C", "H", "S"]
)
atomicdescriptor_onehot = atomicdescriptors(
    "./embedding_onehot.json",
```

```
    overwritten=True,
    element_types=["C", "H", "S"],
    one_hot=True,
)
```

The first argument of the `atomicdescriptors` class must provide the name of the JSON file from which the atomic descriptors can be extracted. If a JSON file with the specified name already exists, the atomic descriptors are loaded from the existing file. If the JSON file does not exist, it is generated within the call to the constructor of the `atomicdescriptors` class.

15. COMPLETE EXAMPLES

This section provides two complete, end-to-end examples of training GNN models with HydraGNN: one using the QM9 molecular dataset and one using the FePt alloy dataset produced by LSMS calculations. Each example demonstrates both single-task and multi-task training configurations.

15.1 QM9 MOLECULAR DATASET

The QM9 dataset (Ramakrishnan et al. 2014; Wu et al. 2018) is a popular benchmark for molecular property prediction. It contains approximately 134,000 organic molecules, each with up to 9 heavy atoms (C, N, O, F) plus hydrogen. Each molecule is annotated with 19 properties computed using quantum chemistry methods. A summary of the QM9 properties used in HydraGNN is provided in Table 9.

Table 9. Summary of the QM9 molecular properties.

Index	Property	Description	Unit
0	μ	Dipole moment	Debye
1	α	Isotropic polarizability	Bohr^3
2	ϵ_{HOMO}	HOMO energy	Hartree
3	ϵ_{LUMO}	LUMO energy	Hartree
4	$\Delta\epsilon$	HOMO-LUMO gap	Hartree
5	$\langle R^2 \rangle$	Electronic spatial extent	Bohr^2
6	ZPVE	Zero-point vibrational energy	Hartree
7	U_0	Internal energy at 0 K	Hartree
8	U	Internal energy at 298 K	Hartree
9	H	Enthalpy at 298 K	Hartree
10	G	Free energy at 298 K	Hartree
11	c_v	Heat capacity at 298 K	cal/(mol K)

15.1.1 General Workflow

The general workflow for any GNN training task in HydraGNN involves:

1. Load the raw data and define atomic descriptors
2. Transform data into `torch_geometric.data.Data` objects
3. Split the dataset into training, validation, and test subsets
4. Define the GNN model architecture and training hyperparameters
5. Train the model and monitor learning via the validation loss
6. Evaluate the trained model on the held-out test set

The complete source code for this example is in `examples/qm9/qm9.py` with its JSON configuration in `examples/qm9/qm9.json`.

15.1.2 Single-Task Training

In single-task mode, HydraGNN trains one model to predict a single target property. The current QM9 example predicts the free energy (G) per atom at 298 K using a GraphGPS (General, Powerful, and Scalable Graph Transformer) architecture with SchNet as the local MPNN backbone.

15.1.2.1 Loading and Transforming QM9.

HydraGNN loads the QM9 dataset via PyTorch Geometric's `QM9` dataset class. The `pre_transform` function converts each molecule into HydraGNN format, computes Laplacian positional encodings (LPE) for GraphGPS, and sets up graph-level attributes for conditioning:

Listing 29. Pre-transform function for QM9 (from `examples/qm9/qm9.py`)

```
from torch_geometric.datasets import QM9
from torch_geometric.transforms import (
    AddLaplacianEigenvectorPE,
)

charge, spin = 0.0, 1.0  # constant for QM9

transform = AddLaplacianEigenvectorPE(
    k=config["NeuralNetwork"]["Architecture"]["pe_dim"],
    attr_name="pe", is_undirected=True,
)

def qm9_pre_transform(data, transform):
    data = transform(data)        # compute LPE
    data.x = data.z.float().view(-1, 1)  # atomic number
    data.y = data.y[:, 10] / len(data.x) # per-atom G
    data.graph_attr = torch.tensor(
        [charge, spin], dtype=torch.float32)
    # relative LPE as edge features for GPS
    source_pe = data.pe[data.edge_index[0]]
    target_pe = data.pe[data.edge_index[1]]
    data.rel_pe = torch.abs(source_pe - target_pe)
    return data

dataset = QM9(
    root="dataset/qm9",
    pre_transform=lambda d: qm9_pre_transform(d, transform),
    pre_filter=lambda d: d.idx < num_samples,
)
```

Note that the target property is divided by the number of atoms (`len(data.x)`) to produce per-atom free energy, which typically improves learning on molecules of varying sizes. The `pre_filter` limits the dataset to a small subset (`num_samples` = 1000) for quick demonstration; remove it for production runs.

15.1.2.2 Model Architecture and Training Configuration.

The JSON configuration uses the GPS transformer with SchNet as the local message-passing backbone, Laplacian positional encodings of dimension 2, multi-head attention with 8 heads, and an AdamW optimizer with a ReduceLROnPlateau scheduler:

Listing 30. JSON configuration for single-task QM9 training (from `qm9.json`)

```
{
  "NeuralNetwork": {
    "Architecture": {
      "global_attn_engine": "GPS",
      "global_attn_type": "multihead",
```

```
      "mpnn_type": "SchNet",
      "radius": 7,
      "max_neighbours": 5,
      "pe_dim": 2,
      "global_attn_heads": 8,
      "hidden_dim": 64,
      "num_conv_layers": 2,
      "num_gaussians": 10,
      "num_filters": 8,
      "output_heads": {
        "graph": {
          "num_sharedlayers": 2,
          "dim_sharedlayers": 5,
          "num_headlayers": 2,
          "dim_headlayers": [50, 25]
        }
      },
      "task_weights": [1.0]
    },
    "Training": {
      "num_epoch": 2,
      "perc_train": 0.7,
      "loss_function_type": "mse",
      "batch_size": 64,
      "Optimizer": {
        "type": "AdamW",
        "learning_rate": 1e-3
      }
    }
  }
}
```

15.1.2.3 Training Pipeline.

The training script follows the standard HydraGNN pipeline. Key steps include initializing DDP, splitting the dataset, creating data loaders, updating the configuration (which auto-populates derived fields such as `edge_dim` and `num_nodes`), building the model, wrapping it for distributed training, and running the training loop:

Listing 31. QM9 training pipeline (simplified from `qm9.py`)

```
import hydragnn

# Initialize distributed training
world_size, world_rank = (
    hydragnn.utils.distributed.setup_ddp()
)

# Split and create data loaders
train, val, test = hydragnn.preprocess.split_dataset(
    dataset, config["NeuralNetwork"]["Training"]
    ["perc_train"], False)
train_loader, val_loader, test_loader = (
    hydragnn.preprocess.create_dataloaders(
```

```
        train, val, test,
        config["NeuralNetwork"]["Training"]["batch_size"]
    )
)

# Update config with dataset-derived values
config = hydragnn.utils.input_config_parsing\
    .update_config(
        config, train_loader, val_loader, test_loader)

# Create model and optimizer
model = hydragnn.models.create_model_config(
    config=config["NeuralNetwork"], verbosity=verbosity)
optimizer = torch.optim.AdamW(
    model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", factor=0.5,
    patience=5, min_lr=1e-5)
model, optimizer = (
    hydragnn.utils.distributed
    .distributed_model_wrapper(
        model, optimizer, verbosity)
)

# Train
hydragnn.train.train_validate_test(
    model, optimizer,
    train_loader, val_loader, test_loader,
    writer, scheduler,
    config["NeuralNetwork"], log_name, verbosity)
```

To run the example:

Listing 32. Running the QM9 example

```
cd examples/qm9
python qm9.py                    # single GPU
python qm9.py --mpnn_type PNA    # override model type
torchrun --nproc_per_node=4 qm9.py  # 4-GPU DDP
```

15.1.3 Multi-Task Training

In multi-task mode, HydraGNN's multi-head architecture can simultaneously predict multiple properties of each molecule. This example demonstrates training on all 12 graph-level QM9 properties at once.

The main differences from the single-task configuration are:

- The `pre_transform` function assigns all 12 target properties as output values
- The JSON configuration specifies 12 graph-level output heads, each with its own MLP branch
- The loss function combines contributions from all tasks, improving generalization through shared representations

Listing 33. Pre-transform function for QM9 multi-task training (12 properties)

```
def qm9_pre_transform_multi(data):
    # Select all 12 graph-level QM9 properties
    data.y = data.y[:, 0:12]
    graph_features_dim = [1] * 12
    node_feature_dim = [1] * 12
    update_predicted_values(
        12, 12, graph_features_dim, node_feature_dim, data
    )
    update_atom_features(data, ["atomic_number"])
    return data
```

The JSON configuration for multi-task training replicates the output head specification across all 12 target properties. Each head shares the same intermediate layers of the GNN backbone but has its own decoding MLP.

15.2 FEPT ALLOY DATASET (LSMS)

The FePt alloy dataset (Lupo Pasini and Eisenbach 2021) provides atomic-level quantities computed with the locally self-consistent multiple scattering (LSMS) first-principles electronic structure code. This is a mixed graph-level and node-level prediction problem, combining a graph-level property (free energy scaled by the number of nodes) with per-atom properties (charge density and magnetic moment).

The dataset consists of 1,024 configurations of a 32-atom FePt crystalline system at varying compositions and temperatures. Each configuration includes:

- Atomic positions and types (Fe or Pt)
- Charge density at each atomic site
- Magnetic moment at each atomic site
- Total, band, and Fermi energies

The complete source code is in `examples/lsms/lsms.py` with its JSON configuration in `examples/lsms/lsms.json`.

The dataset can be downloaded from the OLCF data portal via Globus:

Listing 34. Obtaining the FePt dataset via Globus

```
# Collection: OLCF DTNV public endpoint
# Path: /projects/datascience/HydraGNN_data/FePt32_randomized
```

15.2.1 Data Loading and Serialization

Unlike the QM9 example, the LSMS example uses HydraGNN's custom `LSMSDataset` loader and a two-phase workflow: first preprocess and serialize the data, then load the serialized data for training. The dataset specification in the JSON config defines the raw data format and the features to extract:

Listing 35. Dataset section of `lsms.json`

```
"Dataset": {
    "name": "FePt_32atoms",
    "path": {"total": "./dataset/FePt_enthalpy"},
    "format": "LSMS",
```

```
    "compositional_stratified_splitting": true,
    "node_features": {
        "name": ["num_of_protons",
                 "charge_density",
                 "magnetic_moment"],
        "dim": [1, 1, 1],
        "column_index": [0, 5, 6]
    },
    "graph_features": {
        "name": ["free_energy_scaled_num_nodes"],
        "dim": [1],
        "column_index": [0]
    }
}
```

The `compositional_stratified_splitting` flag ensures that the train/validation/test split preserves the composition distribution across subsets, which is important for alloy datasets with varying Fe/Pt ratios.

Data is serialized to either pickle or ADIOS format for efficient distributed loading:

Listing 36. Preprocessing and serialization (from `lsms.py`)

```
from hydragnn.utils.datasets.lsmsdataset import LSMSDataset
from hydragnn.utils.datasets.serializeddataset import (
    SerializedWriter, SerializedDataset)
from hydragnn.preprocess.load_data import split_dataset

# Phase 1: preprocess (rank 0 only)
total = LSMSDataset(config)
trainset, valset, testset = split_dataset(
    total, perc_train=0.7,
    stratify_splitting=True)
SerializedWriter(trainset, basedir, datasetname,
    "trainset",
    minmax_node_feature=total.minmax_node_feature,
    minmax_graph_feature=total.minmax_graph_feature)
SerializedWriter(valset, basedir, datasetname,
    "valset")
SerializedWriter(testset, basedir, datasetname,
    "testset")

# Phase 2: load serialized data (all ranks)
trainset = SerializedDataset(
    basedir, datasetname, "trainset")
valset = SerializedDataset(
    basedir, datasetname, "valset")
testset = SerializedDataset(
    basedir, datasetname, "testset")
```

The `minmax_node_feature` and `minmax_graph_feature` values are computed during preprocessing and stored with the training set. These are used for output denormalization during evaluation when `"denormalize_output": true` is set in the configuration.

15.2.2 Multi-Task Training on FePt

The FePt example is inherently multi-task, predicting one graph-level property (free energy) and two node-level properties (charge density and magnetic moment) simultaneously. The architecture uses both graph and node output heads:

Listing 37. Architecture and Variables of Interest for multi-task FePt (from `lsms.json`)

```
"Architecture": {
    "mpnn_type": "PNA",
    "radius": 7,
    "max_neighbours": 100,
    "hidden_dim": 5,
    "num_conv_layers": 6,
    "output_heads": {
        "graph": {
            "num_sharedlayers": 2,
            "dim_sharedlayers": 5,
            "num_headlayers": 2,
            "dim_headlayers": [50, 25]
        },
        "node": {
            "num_headlayers": 2,
            "dim_headlayers": [50, 25],
            "type": "mlp"
        }
    },
    "task_weights": [1.0, 1.0, 1.0]
},
"Variables_of_interest": {
    "input_node_features": [0],
    "output_names": [
        "free_energy_scaled_num_nodes",
        "charge_density",
        "magnetic_moment"],
    "output_index": [0, 1, 2],
    "type": ["graph", "node", "node"],
    "denormalize_output": true
}
```

Key points:

- The `type` array specifies `["graph", "node", "node"]`, indicating that the first output is a graph-level property while the second and third are node-level.
- `task_weights` controls the relative importance of each task in the combined loss function. Equal weights are used here.
- `denormalize_output: true` restores predictions to physical units using the stored min/max normalization factors.
- `input_node_features: [0]` uses only the first node feature (number of protons) as input; the remaining features (charge density, magnetic moment) are prediction targets.

As illustrated in Figure 8, the model shares the message-passing backbone while maintaining separate decoding heads for each target property.

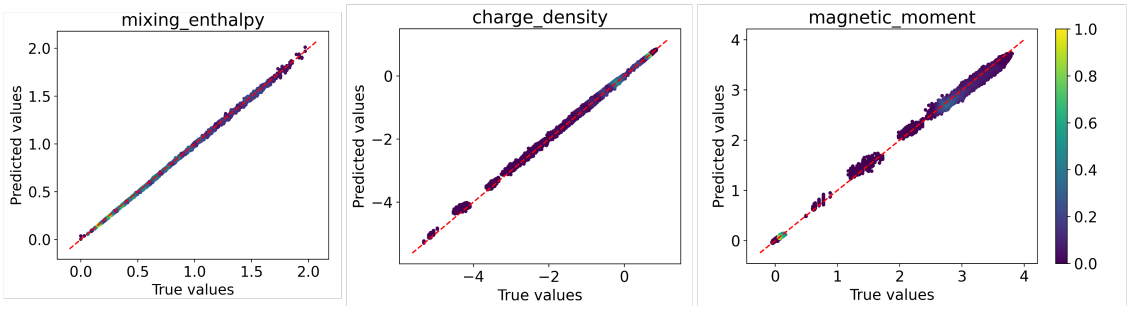

Figure 8. Multi-task architecture for predicting charge density and magnetic moment.

15.2.3 Results and Evaluation

After training, the model can be evaluated by comparing predicted and actual values on the held-out test set. Through inference, the trained model generates predictions that can be visualized using parity plots. A parity plot of test-set predictions for the free energy using the PNA model on FePt is shown in Figure 9.

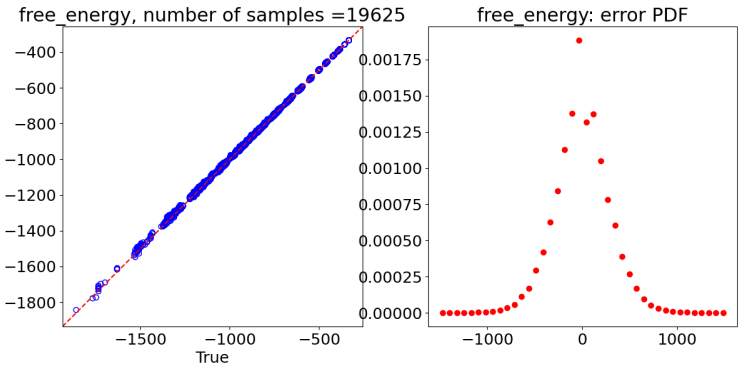

Figure 9. Parity plot of predicted vs. actual free energy values on the FePt test set using PNA.

An example parity plot for scatter predictions is shown in Figure 10. Additionally, Figure 11 shows the formation enthalpy predictions across different compositions.

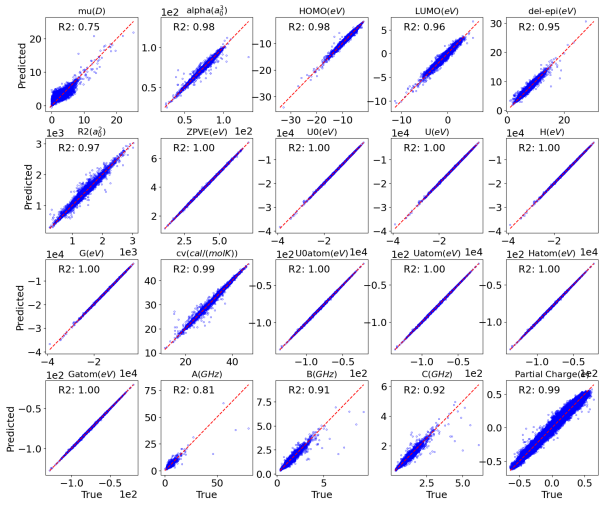

Figure 10. Scatter plot of test-set predictions.

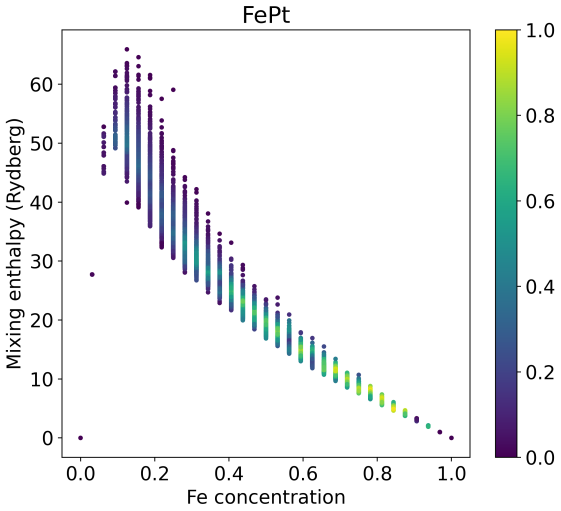

Figure 11. Formation enthalpy predictions.

To run the LSMS/FePt example:

Listing 38. Running the LSMS/FePt example

```
cd examples/lsms
# Phase 1: preprocess and serialize
python lsms.py --preonly --pickle
# Phase 2: train (loads serialized data)
python lsms.py --loadexistingsplit --pickle
# With ADIOS format (for large-scale distributed I/O)
python lsms.py --preonly --adios
mpirun -np 4 python lsms.py --loadexistingsplit --adios
```

15.3 LENNARD-JONES MLIP EXAMPLE

The Lennard-Jones (LJ) example demonstrates how to train a machine-learned interatomic potential (MLIP) using HydraGNN's built-in MLIP mode. Unlike the previous examples that predict various properties from fixed atomic configurations, an MLIP predicts total energy. The MLIP atomic forces are computed as the negative gradient of the total energy with respect to atomic positions, enabling molecular dynamics simulations that conserve energy.

The complete source code is in `examples/LennardJones/` with the main script `LennardJones.py`, data generation in `LJ_data.py`, and the JSON configuration in `LJ.json`.

15.3.1 MLIP Mode Overview

HydraGNN's MLIP mode is activated by setting `"enable_interatomic_potential": true` in the Architecture configuration. In this mode:

1. The model predicts a per-atom energy contribution at each node.
2. The total energy is obtained by summing the per-atom energies over all nodes in the graph.
3. Atomic forces are computed as the negative gradient of the total energy with respect to atomic positions: $\mathbf{F}_i = -\nabla_{\mathbf{r}_i} E_{\text{total}}$.

4. The loss function combines energy and force errors with configurable weights.

The loss function is:

$$\mathcal{L} = w_E \mathcal{L}_E + w_{E/N} \mathcal{L}_{E/N} + w_F \mathcal{L}_F \tag{31}$$

where w_E, $w_{E/N}$, and w_F are the `energy_weight`, `energy_peratom_weight`, and `force_weight`, respectively. $\mathcal{L}_E$ is the error on total energy, $\mathcal{L}_{E/N}$ on per-atom energy, and $\mathcal{L}_F$ on forces.

15.3.2 JSON Configuration for MLIP

The JSON configuration for the LJ MLIP example uses DimeNet with periodic boundary conditions:

Listing 39. JSON configuration for Lennard-Jones MLIP (from `LJ.json`)

```
{
  "NeuralNetwork": {
    "Architecture": {
      "periodic_boundary_conditions": true,
      "mpnn_type": "DimeNet",
      "radius": 5.0,
      "max_neighbours": 5,
      "hidden_dim": 32,
      "num_conv_layers": 4,
      "enable_interatomic_potential": true,
      "energy_weight": 1.0,
      "energy_peratom_weight": 1.0,
      "force_weight": 1.0,
      "num_radial": 5,
      "num_spherical": 2,
      "envelope_exponent": 5,
      "output_heads": {
        "node": {
          "num_headlayers": 2,
          "dim_headlayers": [60, 20],
          "type": "mlp"
        }
      },
      "task_weights": [1]
    },
    "Variables_of_interest": {
      "input_node_features": [0],
      "output_index": [0],
      "type": ["node"],
      "output_dim": [1],
      "output_names": ["graph_energy"]
    },
    "Training": {
      "num_epoch": 25,
      "batch_size": 64,
      "perc_train": 0.7,
      "early_stopping": true,
      "patience": 20,
      "Optimizer": {
        "type": "Adam",
        "learning_rate": 0.005
      }
```

```
    }
  }
}
```

Key differences from non-MLIP configurations:

- `enable_interatomic_potential: true` activates the MLIP mode with automatic force computation via backpropagation through the energy.
- `periodic_boundary_conditions: true` uses the `RadiusGraphPBC` class for neighbor finding across periodic images, essential for bulk material simulations.
- The output head is a single `node`-level head that predicts per-atom energy; forces are derived automatically and do not require a separate head.
- `energy_weight`, `energy_peratom_weight`, and `force_weight` balance the three components of the MLIP loss function.

15.3.3 Data Generation and Format

The LJ example generates synthetic training data by sampling atomic configurations and computing energies and forces analytically from the Lennard-Jones potential. The data generation script (`LJ_data.py`) creates configurations with varying numbers of atoms and cell sizes.

Each data sample requires:

- `pos`: atomic positions (Cartesian coordinates)
- `x`: node features (atomic type)
- `cell`: unit cell matrix (3×3) for PBC
- `y`: total energy (graph-level target)
- `forces`: atomic force vectors ($N \times 3$)

15.3.4 Running the MLIP Example

Listing 40. Running the Lennard-Jones MLIP example

```
cd examples/LennardJones
# Preprocess: generate and serialize LJ data
python LennardJones.py --preonly --pickle
# Train the MLIP
python LennardJones.py --loadexistingsplit --pickle
# Override model type (e.g., EGNN, SchNet, MACE)
python LennardJones.py --loadexistingsplit --pickle \
    --mpnn_type EGNN
# Multi-GPU training
mpirun -np 4 python LennardJones.py \
    --loadexistingsplit --pickle
```

After training, the model can be used for inference to predict energies and forces on new configurations. The `LJ_inference_plots.py` script demonstrates how to load a trained checkpoint and generate parity plots comparing predicted vs. reference energies and forces.

16. REFERENCES

Allen, Alice E. A., Nicholas Lubbers, Sakib Matin, Justin Smith, Richard Messerly, Sergei Tretiak, and Kipton Barros. 2024. "Learning together: Towards foundation models for machine learning interatomic potentials with meta-learning." *npj Computational Materials* 10:154. https://doi.org/10.1038/s41524-024-01066-9.

Balaprakash, Prasanna, Michael Salim, Thomas D. Uram, Venkatram Vishwanath, and Stefan M. Wild. 2018. "DeepHyper: Asynchronous Hyperparameter Search for Deep Neural Networks." In *2018 IEEE 25th International Conference on High Performance Computing (HiPC),* 42–51. https://doi.org/10.1109/HiPC.2018.00014.

Barroso-Luque, Luis, Muhammed Shuaibi, Xiang Fu, Brandon M. Wood, et al. 2024. "Open Materials 2024 (OMat24) Inorganic Materials Dataset and Models." *arXiv preprint arXiv:2410.12771.*

Batatia, Ilyes, Philipp Benner, Yuan Chiang, Alin M. Elena, et al. 2024. *A Foundation Model for Atomistic Simulations of the Elements.* arXiv: 2401.00096 [physics.chem-ph].

Batatia, Ilyes, Dávid P. Kovács, Gregor N. C. Simm, Christoph Ortner, and Gábor Csányi. 2022. "MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields." *arXiv preprint,* arXiv: 2206.07697 [stat.ML]. https://doi.org/10.48550/arXiv.2206.07697.

Batzner, Simon, Albert Musaelian, Lixin Sun, and et al. 2022. "E(3)-equivariant Graph Neural Networks for Data-efficient and Accurate Interatomic Potentials." *Nature Communications* 13:2453. https://doi.org/10.1038/s41467-022-29939-5.

Chanussot, Lowik, Abhishek Das, Siddhant Goyal, Thibault Lavril, Muhammed Shuaibi, et al. 2021. "Open Catalyst 2020 (OC20) Dataset and Community Challenges." *ACS Catalysis* 11 (10): 6059–6072. https://doi.org/10.1021/acscatal.0c04525.

Choi, Jong Youl, Massimiliano Lupo Pasini, Pei Zhang, Kshitij Mehta, Frank Liu, Jonghyun Bae, and Khaled Ibrahim. 2023. "DDStore: Distributed Data Store for Scalable Training of Graph Neural Networks on Large Atomistic Modeling Datasets." In *SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis.* https://doi.org/10.1145/3624062.3624171.

Dinan, James, Pavan Balaji, Darius Buntinas, David Goodell, William Gropp, and Rajeev Thakur. 2016. "An implementation and evaluation of the MPI 3.0 one-sided communication interface." *Concurrency and Computation: Practice and Experience* 28 (17): 4385–4404.

Fuchs, Fabian B., Daniel E. Worrall, Volker Fischer, and Max Welling. 2020. "SE(3)-Transformers: 3D Roto-Translation Equivariant Attention Networks." *arXiv preprint arXiv:2006.10503,* arXiv: 2006.10503 [cs.LG]. https://doi.org/10.48550/arXiv.2006.10503.

Gabriel, Edgar, Graham E Fagg, George Bosilca, Thara Angskun, Jack J Dongarra, Jeffrey M Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, et al. 2004. "Open MPI: Goals, concept, and design of a next generation MPI implementation." In *Recent Advances in Parallel Virtual Machine and Message Passing Interface,* 97–104. Springer.

Gasteiger, Johannes, Janek Groß, and Stephan Günnemann. 2022. *Directional Message Passing for Molecular Graphs.* arXiv: 2003.03123 [cs.LG]. https://arxiv.org/abs/2003.03123.

Gilmer, Justin, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. "Neural Message Passing for Quantum Chemistry." In *Proceedings of the 34th International Conference on Machine Learning,* edited by Doina Precup and Yee Whye Teh, 70:1263–1272. Proceedings of Machine Learning Research. PMLR, June. https://proceedings.mlr.press/v70/gilmer17a.html.

Godoy, William F., Norbert Podhorszki, Ruonan Wang, Chuck Atkins, Greg Eisenhauer, Junmin Gu, Philip Davis, Jong Choi, Kai Germaschewski, Kevin Huck, et al. 2020. "ADIOS 2: The Adaptable Input Output System. A framework for high-performance data management." *SoftwareX* 12:100561.

Gropp, William, Ewing Lusk, Nathan Doss, and Anthony Skjellum. 1996. "A high-performance, portable implementation of the MPI message passing interface standard." *Parallel Computing* 22 (6): 789–828.

Higham, Nicholas J. 2002. *Accuracy and Stability of Numerical Algorithms.* 2nd ed. Society for Industrial / Applied Mathematics. https://doi.org/10.1137/1.9780898718027.

Hoja, Johannes, Leonardo Medrano Sandonas, Brian G. Ernst, Alvaro Vazquez-Mayagoitia, Robert A. DiStasio Jr., and Alexandre Tkatchenko. 2021. "QM7-X, a comprehensive dataset of quantum-mechanical properties spanning the chemical space of small organic molecules." *Scientific Data* 8:43. https://doi.org/10.1038/s41597-021-00812-2.

Jacobson, Leif, James Stevenson, Farhad Ramezanghorbani, Steven Dajnowicz, and Karl Leswing. 2023. "Leveraging Multitask Learning to Improve the Transferability of Machine Learned Force Fields." Preprint, *ChemRxiv,* https://doi.org/10.26434/chemrxiv-2023-8n737.

Jain, Anubhav, Shyue Ping Ong, Geoffroy Hautier, Wei Chen, William Davidson Richards, Stephen Dacek, Shreyas Cholia, et al. 2013. "Commentary: The Materials Project: A materials genome approach to accelerating materials innovation." *APL Materials* 1 (1): 011002. https://doi.org/10.1063/1.4812323.

Kipf, Thomas N., and Max Welling. 2017. "Semi-Supervised Classification with Graph Convolutional Networks." *arXiv preprint arXiv:1609.02907,* https://doi.org/10.48550/arXiv.1609.02907. arXiv: 1609.02907 [cs.LG].

Kong, Lingyu, Jaeheon Shim, Guoxiang Hu, and Victor Fung. 2026. "Scalable foundation interatomic potentials via message-passing pruning and graph partitioning." *npj Computational Materials,* https://doi.org/10.1038/s41524-026-02001-4.

Levine, Daniel S., Nicholas Liesen, Lauren Chua, et al. 2025. "The Open Polymers 2026 (OPoly26) Dataset and Evaluations." *arXiv preprint arXiv:2512.23117.*

Li, Shen, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. "PyTorch Distributed: Experiences on accelerating data parallel training." *arXiv preprint arXiv:2006.15704.*

Lubbers, Nicholas, Justin S. Smith, and Kipton Barros. 2018. "Hierarchical modeling of molecular energies using a deep neural network." *The Journal of Chemical Physics* 148, no. 24 (March): 241715. ISSN: 0021-9606. https://doi.org/10.1063/1.5011181. eprint: https://pubs.aip.org/aip/jcp/article-pdf/doi/10.1063/1.5011181/16654049/241715_1_online.pdf. https://doi.org/10.1063/1.5011181.

Lupo Pasini, M., and M. Eisenbach. 2021. "FePt binary alloy with 32 atoms – LSMS-3 data" (February). https://doi.org/10.13139/OLCF/1762742.

Lupo Pasini, Massimiliano, Jong Youl Choi, Kshitij Mehta, Pei Zhang, David Rogers, Jonghyun Bae, Khaled Z. Ibrahim, et al. 2025. "Scalable training of trustworthy and energy-efficient predictive graph foundation models for atomistic materials modeling: a case study with HydraGNN." *Journal of Supercomputing* 81:Article 618.

Md, Vasimuddin, Sanchit Misra, Guixiang Ma, Ramanarayan Mohanty, Evangelos Georganas, Alexander Heinecke, Dhiraj Kalamkar, Nesreen K. Ahmed, and Sasikanth Avancha. 2021. "DistGNN: Scalable Distributed Training for Large-Scale Graph Neural Networks." In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21).* https://doi.org/10.1145/3458817.3480856.

mendeleev. https://mendeleev.readthedocs.io/en/stable/.

Merchant, A., S. Batzner, S. S. Schoenholz, and et al. 2023. "Scaling deep learning for materials discovery." Published online 29 November 2023, issue date 07 December 2023, *Nature* 624:80–85. https://doi.org/10.1038/s41586-023-06735-9.

Micikevicius, Paulius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, et al. 2018. "Mixed Precision Training." In *International Conference on Learning Representations (ICLR).*

Microsoft. 2020. *DeepSpeed.* https://github.com/microsoft/DeepSpeed. Open-source deep learning optimization library.

Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. "PyTorch: An imperative style, high-performance deep learning library." *Advances in Neural Information Processing Systems* 32.

R. Caruana. 1993. "Multitask learning: a knowledge-based source of inductive bias." *Mach. Learn.* 48:41–48. https://doi.org/10.1023/A:1007379606734.

Rajbhandari, Samyam, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models." *arXiv preprint arXiv:1910.02054.*

Ramakrishnan, Raghunathan, Pavlo O Dral, Matthias Rupp, and O Anatole Von Lilienfeld. 2014. "Quantum chemistry structures and properties of 134 kilo molecules." *Scientific Data* 1 (1): 1–7.

Rampášek, Ladislav, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2022. "Recipe for a General, Powerful, Scalable Graph Transformer." In *Advances in Neural Information Processing Systems (NeurIPS),* vol. 35. https://arxiv.org/abs/2205.12454.

Sahoo, Sushree Jagriti, Mikael Maraschin, Daniel S. Levine, Zachary Ulissi, C. Lawrence Zitnick, Joel B. Varley, and Joseph A. Gauthier. 2025. "The Open Catalyst 2025 (OC25) Dataset and Models for Solid-Liquid Interfaces." *arXiv preprint arXiv:2509.17862.*

Schmidt, Jonathan, Love Pettersson, Claudio Verdozzi, Silvana Botti, and Miguel A. L. Marques. 2021. "Crystal graph attention networks for the prediction of stable materials." *Science Advances* 7 (49): eabi7948. https://doi.org/10.1126/sciadv.abi7948.

Shiota, Tomoya, Kenji Ishihara, Tuan Minh Do, Toshio Mori, and Wataru Mizukami. 2024. *Taming Multi-Domain, -Fidelity Data: Towards Foundation Models for Atomistic Scale Simulations.* arXiv: 2412.13088 [physics.chem-ph].

Shoeybi, Mohammad, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism." *arXiv preprint arXiv:1909.08053*.

Smith, Justin S., Roman Zubatyuk, Benjamin Nebgen, Nicholas Lubbers, Kipton Barros, Adrian E. Roitberg, Olexandr Isayev, and Sergei Tretiak. 2020. "The ANI-1ccx and ANI-1x data sets, coupled-cluster and density functional theory properties for molecules." *Scientific Data* 7:134. https://doi.org/10.1038/s41597-020-0473-z.

Sypetkowski, Maciej, Frederik Wenkel, Farimah Poursafaei, Nia Dickson, Karush Suri, Philip Fradkin, and Dominique Beaini. 2024. "On the Scalability of GNNs for Molecular Graphs." In *Advances in Neural Information Processing Systems 38*.

Takamoto, So, Chikashi Shinagawa, Daisuke Motoki, et al. 2022. "Towards universal neural network potential for material discovery applicable to arbitrary combination of 45 elements." *Nature Communications* 13:2991. https://doi.org/10.1038/s41467-022-30700-4.

Tran, Kevin, Muhammed Shuaibi, Abhishek Das, et al. 2023. "Open Catalyst 2022 (OC22) Dataset and Challenges for Oxidation Electrocatalysts." *ACS Catalysis* 13 (5): 3066–3084. https://doi.org/10.1021/acscatal.2c05426.

Wu, Zhenqin, Bharath Ramsundar, Evan N Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S Pappu, Karl Leswing, and Vijay Pande. 2018. "MoleculeNet: a benchmark for molecular machine learning." *Chemical Science* 9 (2): 513–530.

Zhang, Duo, Xinzijian Liu, Xiangyu Zhang, et al. 2024. *DPA-2: a large atomic model as a multi-task learner*. arXiv: 2312.15492 [physics.chem-ph].

Zhao, Yanli, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, et al. 2023. "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel." *Proceedings of the VLDB Endowment* 16 (12): 3848–3860. https://doi.org/10.14778/3611540.3611569.

APPENDIX A. SUPPORTED MESSAGE-PASSING POLICIES

APPENDIX A. SUPPORTED MESSAGE-PASSING POLICIES

Table A.1 provides a comprehensive summary of all 13 message-passing policies supported by HydraGNN.

Table A.1. Summary of message-passing policies supported in HydraGNN

mpnn_type	Model Name	Class	Edge Features	Equivariant	Key Characteristics
GIN	Graph Isomorphism Network	GINStack	No	No	Learnable epsilon, simple aggregation
PNA	Principal Neighborhood Aggr.	PNAStack	Yes	No	Multiple aggregators and scalers
PNAPlus	PNA with Radial Basis	PNAPlusStack	Yes	No	PNA + Bessel radial basis functions
GAT	Graph Attention Network	GATStack	Yes	No	Multi-head attention (GATv2Conv)
MFC	Molecular Fingerprint Conv.	MFCStack	No	No	Max degree parameter
CGCNN	Crystal Graph CNN	CGCNNStack	Yes	No	Designed for crystal structures
SAGE	GraphSAGE	SAGEStack	No	No	Simple neighborhood aggregation
SchNet	Continuous-filter Conv.	SCFStack	Yes	Optional	Gaussian smearing, ShiftedSoftplus
DimeNet	Directional Message Passing	DIMEStack	Yes	No	Bessel + spherical basis, triplet info
EGNN	E(n) Equiv. GNN	EGCLStack	Yes	Toggleable	Position updates, E(n) equivariance
PNAEq	Equivariant PNA	PNAEqStack	Yes	Yes	PNA + PAINN-style equivariant updates
PAINN	Polarizable Atom Interaction	PAINNStack	Yes	Yes	Scalar + vector features, Bessel RBF
MACE	Multi Atomic Cluster Expansion	MACEStack	Yes	Yes (O(3))	Spherical harmonics, e3nn, n-body

APPENDIX B. SUPPORTED LOSS FUNCTIONS, ACTIVATIONS, AND OPTIMIZERS

APPENDIX B. SUPPORTED LOSS FUNCTIONS, ACTIVATIONS, AND OPTIMIZERS

B.1 LOSS FUNCTIONS

The following loss functions are supported via the `loss_function_type` key:

- `mse`: Mean Squared Error
- `mae`: Mean Absolute Error (L1 Loss)
- `smooth_l1`: Smooth L1 Loss (Huber Loss)
- `rmse`: Root Mean Squared Error
- `GaussianNLLLoss`: Gaussian Negative Log-Likelihood (for uncertainty quantification; model outputs both mean and variance)

B.2 ACTIVATION FUNCTIONS

The following activation functions are supported via the `activation_function` key:

- `relu`, `selu`, `prelu`, `elu`
- `lrelu_01` (Leaky ReLU, slope 0.1), `lrelu_025` (slope 0.25), `lrelu_05` (slope 0.5)
- `sigmoid`

B.3 OPTIMIZERS

The following optimizers are supported via the `Optimizer.type` key. Each optimizer is also available in a zero-redundancy variant for memory-efficient distributed training:

- `SGD`, `Adam`, `AdamW`, `Adadelta`, `Adagrad`, `Adamax`, `RMSprop`
- `FusedLAMB` (requires DeepSpeed)